

BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÁO CÁO CUỐI KÌ

**Thiết kế và xây dựng ứng dụng quản lý mật khẩu bảo mật dựa
trên cơ chế mã hóa phía máy khách và xác thực blockchain**

Giảng viên hướng dẫn:	TS. Kim Ngọc Bách
Họ và tên sinh viên:	Hoàng Tiến Đạt
Mã sinh viên:	B23DCCE015
Lớp:	D23CQCE06-B
Nhóm:	01

HÀ NỘI - 2026

Mục lục

1	Giới thiệu Dự án	1
1.1	Lý do chọn đề tài	1
1.2	Mục tiêu nghiên cứu	1
1.3	Ý nghĩa	2
1.3.1	Giá trị học thuật (Ý nghĩa khoa học)	2
1.3.2	Tính ứng dụng (Ý nghĩa thực tiễn)	2
2	Cơ sở lý thuyết và công nghệ sử dụng	3
2.1	Cơ sở lý thuyết	3
2.1.1	Mật mã học ứng dụng trong bảo vệ mật khẩu	3
2.1.2	Kiến trúc mã hóa phía máy khách (Client-side Encryption)	4
2.1.3	Cơ chế định danh bằng google	4
2.1.4	Lưu trữ kết hợp với IPFS và xác thực bằng blockchain	5
2.1.5	Công cụ phân tích dựa trên mô hình ước lượng entropy thực nghiệm	5
2.2	Công nghệ sử dụng	5
2.2.1	Nền tảng Blockchain	5
2.2.2	Frontend	6
2.2.3	Mạng lưới lưu trữ phân tán IPFS	6
2.2.4	Công nghệ lưu trữ cục bộ IndexedDB	7
2.2.5	Công cụ đánh giá mật khẩu	7
3	Phân tích và thiết kế hệ thống	8
3.1	Xác định các Tác nhân (Actors)	8
3.2	Sơ đồ usecase Tổng quát	9
3.3	Chi tiết usecase	9
3.3.1	Tổng hợp danh sách các ca sử dụng (Use Cases)	9
3.3.2	UC1: Đăng nhập	10
3.3.3	UC2: Đăng ký	11
3.3.4	UC3: Thêm mật khẩu	13

3.3.5	UC4: Sửa mật khẩu	13
3.3.6	UC5: Xóa mật khẩu	14
3.3.7	UC6: Tạo mật khẩu mạnh ngẫu nhiên	14
3.3.8	UC7: Xuất dữ liệu	15
3.3.9	UC8: Nhập dữ liệu	15
3.3.10	UC9: Xóa dữ liệu	16
3.3.11	UC10: Báo cáo bảo mật	16
3.3.12	UC11: Đổi master password	17
3.3.13	UC12: Cập nhật Thông tin cá nhân	17
3.4	Thiết kế kiến trúc hệ thống	18
3.4.1	Thiết kế chi tiết Tầng Lưu trữ (Storage Layer)	20
3.4.2	Thiết kế chi tiết Tầng Nghiệp vụ và Bảo mật (Business Logic & Security Layer)	23
3.4.3	Thiết kế chi tiết Tầng Giao diện (Frontend / Presentation Layer)	24
3.4.4	Thiết kế hệ thống giao diện lập trình ứng dụng (API Design)	25
3.5	Đặc tả chi tiết danh mục chức năng API (API Directory)	25
3.5.1	Danh mục chức năng API theo các phân tầng kiến trúc	25
3.5.2	Đặc tả các giao diện lập trình trình duyệt tích hợp (Browser Native APIs)	29
3.5.3	Giao diện lập trình ứng dụng HTTP Web API (JIT Gas Faucet Server)	31
3.5.4	Đặc tả các giao diện lập trình thư viện ngoài tích hợp (External Library APIs)	33
3.6	Thiết kế tĩnh của hệ thống (Static Design)	34
3.7	Đặc tả chi tiết các mô hình và cấu trúc dữ liệu hệ thống	34
3.8	Thiết kế hệ thống biểu đồ tuần tự (Sequence Diagrams)	39
4	Kết quả cài đặt và thử nghiệm	46
4.1	Môi trường cài đặt và triển khai	46
4.1.1	Môi trường phần cứng	46
4.1.2	Môi trường phần mềm	46
4.1.3	Công nghệ và thư viện chính	47
4.2	Kết quả cài đặt hệ thống	47
4.2.1	Cài đặt và khởi chạy ứng dụng	47
4.2.2	Triển khai Smart Contract trên Blockchain	49
4.3	Kết quả giao diện người dùng	50
4.3.1	Trang xác thực đăng nhập (AuthPage)	50
4.3.2	Trang thiết lập Master Password	51
4.3.3	Trang kết sắt quản lý mật khẩu (VaultPage)	52
4.3.4	Trang sinh mật khẩu ngẫu nhiên (GeneratorPage)	53
4.3.5	Trang báo cáo an toàn (ReportsPage)	54

4.3.6	Trang cài đặt bảo mật (SecurityPage)	54
4.3.7	Trang nhập/xuất dữ liệu (ImportExportPage)	55
4.4	Kết quả thử nghiệm chức năng	56
4.4.1	Thử nghiệm mã hóa cục bộ và IndexedDB	56
4.4.2	Thử nghiệm giao diện và thao tác CRUD	57
4.4.3	Thử nghiệm đánh giá độ mạnh mật khẩu	57
4.4.4	Thử nghiệm kết nối MetaMask	59
4.4.5	Thử nghiệm đăng nhập Google và sinh ví ảo tự động	60
4.4.6	Thử nghiệm tải lên và tải xuống dữ liệu IPFS	60
4.4.7	Thử nghiệm ghi và đọc CID trên Blockchain	61
4.4.8	Thử nghiệm đồng bộ xuyên thiết bị (Cross-Device Sync)	62
4.4.9	Thử nghiệm tính năng tự động khóa (Auto-Lock)	63
4.4.10	Thử nghiệm đổi Master Password (Key Rotation)	64
4.4.11	Thử nghiệm nhập/xuất dữ liệu (Import/Export)	64
4.5	Thử nghiệm xử lý lỗi và tình huống ngoại lệ	65
4.6	Thử nghiệm hiệu năng	67
4.6.1	Hiệu năng mã hóa và giải mã	67
4.6.2	Hiệu năng đồng bộ Web3	68
4.7	Thử nghiệm bảo mật	68
4.7.1	Kiểm tra an toàn dữ liệu trên đường truyền	68
4.7.2	Phân tích mô hình đe dọa (Threat Model)	69
4.8	Tổng hợp kết quả thử nghiệm	69
4.9	Đánh giá và nhận xét	70
4.9.1	Ưu điểm	70
4.9.2	Hạn chế	71
4.9.3	Hướng phát triển	71

5 Kết luận

Danh sách bảng

3.13	Đặc tả cấu trúc thực thể VaultItem trên RAM	35
3.14	Đặc tả cấu trúc thông tin người dùng GoogleUser	36
3.15	Đặc tả các khóa cấu hình trong store vaultMeta	37
4.1	Cấu hình phần cứng môi trường phát triển	46
4.2	Môi trường phần mềm và công cụ phát triển	47
4.3	Ngăn xếp công nghệ chính của hệ thống	47
4.4	Thông tin triển khai Smart Contract VaultPointer	50
4.5	Kết quả thử nghiệm thao tác CRUD trên giao diện	57
4.6	Kết quả thử nghiệm đánh giá cường độ mật khẩu bằng zxcvbn	57
4.7	Kết quả thử nghiệm xử lý lỗi và tình huống ngoại lệ	66
4.8	Kết quả đo hiệu năng mã hóa/giải mã AES-256-GCM với PBKDF2 (310,000 vòng)	67
4.9	Thời gian trung bình cho các bước đồng bộ Web3	68
4.10	Kết quả kiểm tra bảo mật dữ liệu trên đường truyền mạng	68
4.11	Phân tích mô hình đe dọa và biện pháp phòng vệ	69
4.12	Tổng hợp kết quả thử nghiệm các chức năng chính	70

Danh sách hình vẽ

3.1	Sơ đồ usecase tổng quát	9
3.2	Sơ đồ tổng quát kiến trúc phân tầng của hệ thống	19
3.3	Sơ đồ kiến trúc tầng lưu trữ	22
3.4	Sơ đồ kiến trúc tầng nghiệp vụ, bảo mật	24
3.5	Sơ đồ kiến trúc tầng giao diện	25
3.6	Thiết kế biểu đồ tĩnh	34
3.7	Biểu đồ tuần tự luồng hành động Đăng nhập hệ thống	40
3.8	Biểu đồ tuần tự luồng hành động Đăng ký tài khoản mới	40
3.9	Biểu đồ tuần tự luồng hành động Thay đổi mật khẩu chủ	41
3.10	Biểu đồ tuần tự luồng hành động Đồng bộ và Giải mã kết sắt	41
3.11	Biểu đồ tuần tự luồng hành động Thêm bản ghi mật khẩu mới	42
3.12	Biểu đồ tuần tự luồng hành động Khóa phiên làm việc tự động	43
3.13	Biểu đồ tuần tự luồng hành động Tài trợ phí Gas tự động	44
3.14	Biểu đồ tuần tự luồng hành động Kết xuất báo cáo bảo mật	45
4.1	Kết quả biên dịch Solidity triển khai Smart Contract VaultPointer lên mạng Sepolia Testnet	48
4.2	Kết quả triển khai Smart Contract VaultPointer lên mạng Sepolia Testnet	49
4.3	Khởi chạy đồng thời JIT Gas Station Faucet (cổng 3001) và Frontend Vite (cổng 5173)	49
4.4	Triển khai thành công	50
4.5	Giao diện trang xác thực đăng nhập của ứng dụng	51
4.6	Giao diện thiết lập Master Password với đánh giá cường độ zxcvbn	52
4.7	Giao diện trang kho quản lý mật khẩu chính	53
4.8	Giao diện thêm mật khẩu mới vào kết sắt	53
4.9	Giao diện sinh mật khẩu ngẫu nhiên với tùy chỉnh độ dài và tập ký tự	54
4.10	Giao diện trang báo cáo an toàn phân tích mật khẩu	54
4.11	Giao diện trang cài đặt bảo mật	55
4.12	Giao diện nhập/xuất dữ liệu kết sắt	55
4.13	Kết quả thử nghiệm các thao tác CRUD trên giao diện kết sắt	57

4.14	Kết quả đánh giá cường độ mật khẩu: yếu	58
4.15	Kết quả đánh giá cường độ mật khẩu: mạnh)	59
4.16	Kết nối ví MetaMask với ứng dụng DApp Password Manager	60
4.17	Kết quả upload dữ liệu mã hóa lên IPFS qua Pinata API	61
4.18	Giao dịch ghi CID pointer lên Smart Contract VaultPointer thành công trên Sepolia	62
4.19	Kết quả đồng bộ xuyên thiết bị 1	63
4.20	Kết quả đồng bộ xuyên thiết bị 2	63
4.21	Modal mở khóa phiên sau khi tính năng Auto-Lock kích hoạt	64
4.22	Kết quả import dữ liệu mẫu vào kết sắt	65
4.23	Ví dụ xử lý lỗi và phản hồi trên giao diện người dùng	67

Chương 1

Giới thiệu Dự án

1.1 Lý do chọn đề tài

Trong bối cảnh chuyển đổi số và gia tăng các cuộc tấn công mạng, thông tin đăng nhập đã trở thành mục tiêu khai thác phổ biến, chiếm tỷ lệ lớn trong các vụ vi phạm dữ liệu quy mô lớn [12]. Mô hình lưu trữ tập trung truyền thống luôn tiềm ẩn rủi ro về “điểm yếu tập trung” (Single Point of Failure), nơi một sự cố bảo mật đơn lẻ có thể dẫn đến rò rỉ dữ liệu trên diện rộng [17]. Đặc biệt, đánh cắp thông tin đăng nhập (credential theft) vẫn là một trong những nguyên nhân hàng đầu gây ra các vụ xâm phạm an toàn thông tin hiện nay.

Các hệ thống quản lý mật khẩu truyền thống, dù được mã hóa, vẫn yêu cầu người dùng đặt niềm tin vào một bên trung gian quản lý dữ liệu. Điều này đặt ra yêu cầu cấp thiết về một kiến trúc bảo mật giảm thiểu sự phụ thuộc vào bên trung gian và tăng cường quyền kiểm soát dữ liệu cá nhân cho người dùng [17].

Xuất phát từ những hạn chế của mô hình quản lý mật khẩu tập trung, đề tài hướng tới nghiên cứu và xây dựng một ứng dụng quản lý mật khẩu an toàn theo hướng giảm thiểu sự phụ thuộc vào bên trung gian, tăng cường quyền kiểm soát dữ liệu cho người dùng và tích hợp cơ chế hỗ trợ đánh giá bảo mật hiện đại.

1.2 Mục tiêu nghiên cứu

- **Mục tiêu tổng quát:** Nghiên cứu và xây dựng một ứng dụng quản lý mật khẩu theo định hướng tăng cường bảo mật và quyền riêng tư, giúp người dùng chủ động kiểm soát và bảo vệ thông tin đăng nhập trong môi trường số.
- **Mục tiêu cụ thể:** Đề tài tập trung đề xuất và hiện thực hóa mô hình quản lý mật khẩu dựa trên cơ chế mã hóa phía client và xác thực dữ liệu bằng blockchain, với các nội dung

chính sau:

- *Về kỹ thuật*: Triển khai cơ chế mã hóa phía máy khách (Client-side encryption), đảm bảo nhà cung cấp dịch vụ không thể tiếp cận dữ liệu gốc của người dùng.
- *Về lưu trữ*: Xây dựng mô hình lưu trữ kết hợp. Dữ liệu mã hóa được lưu trữ trên mạng lưới phân tán IPFS, đồng thời sử dụng blockchain để lưu trữ giá trị định danh dữ liệu nhằm đảm bảo tính toàn vẹn và khả năng phát hiện thay đổi trái phép.
- *Về tính năng hỗ trợ*: Tích hợp công cụ ước lượng entropy thực nghiệm để đánh giá độ mạnh của mật khẩu, đưa ra các cảnh báo trực quan cho người dùng.
- *Về trải nghiệm*: Xây dựng giao diện thân thiện với hai phương thức xác thực: kết nối ví blockchain (MetaMask) và đăng nhập qua tài khoản xã hội (Google), phù hợp với đa dạng đối tượng người dùng.

1.3 Ý nghĩa

1.3.1 Giá trị học thuật (Ý nghĩa khoa học)

- **Mô hình bảo mật hướng người dùng**: Đề tài nghiên cứu và áp dụng mô hình mã hóa phía client kết hợp với cơ chế xác thực dữ liệu trên blockchain nhằm giảm thiểu sự phụ thuộc vào máy chủ trung tâm và hạn chế rủi ro rò rỉ dữ liệu.
- **Sự kết hợp công nghệ**: Đề tài góp phần phân tích tính khả thi của việc tích hợp công nghệ blockchain vào hệ thống ứng dụng Web truyền thống, trong đó blockchain đóng vai trò đảm bảo tính toàn vẹn dữ liệu thay vì lưu trữ dữ liệu nhạy cảm [3, 17].

1.3.2 Tính ứng dụng (Ý nghĩa thực tiễn)

- **Giải pháp bảo mật cá nhân**: Ứng dụng cung cấp một công cụ hỗ trợ người dùng quản lý mật khẩu an toàn hơn, góp phần giảm thiểu rủi ro từ các kịch bản rò rỉ dữ liệu và tấn công phổ biến [12].
- **Công cụ hỗ trợ nâng cao nhận thức bảo mật**: Thông qua việc tích hợp thư viện đánh giá độ mạnh mật khẩu dựa trên mô hình entropy thực nghiệm [14], hệ thống không chỉ lưu trữ mật khẩu mà còn hỗ trợ người dùng cải thiện thói quen bảo mật.
- **Khả năng mở rộng**: Mô hình được đề xuất có thể mở rộng cho các ứng dụng lưu trữ thông tin nhạy cảm khác như ghi chú bảo mật, quản lý tài liệu cá nhân hoặc các hệ thống xác thực số trong tương lai.

Chương 2

Cơ sở lý thuyết và công nghệ sử dụng

Chương này trình bày các nền tảng lý thuyết phục vụ việc xây dựng ứng dụng quản lý mật khẩu theo định hướng tăng cường bảo mật và quyền riêng tư. Trọng tâm nghiên cứu bao gồm:

- Cơ chế mã hóa phía máy khách.
- Phương pháp dẫn xuất và quản lý khóa an toàn.
- Cơ chế đảm bảo tính toàn vẹn dữ liệu bằng blockchain.

Blockchain và công cụ đánh giá mật khẩu đóng vai trò hỗ trợ trong việc hiện thực hóa kiến trúc bảo mật này.

2.1 Cơ sở lý thuyết

2.1.1 Mật mã học ứng dụng trong bảo vệ mật khẩu

Mật mã học đóng vai trò nền tảng trong các hệ thống bảo mật hiện đại [6]. Trong ứng dụng quản lý mật khẩu, dữ liệu cần được bảo vệ thông qua cơ chế mã hóa đối xứng mạnh nhằm đảm bảo tính bí mật (confidentiality). Thuật toán AES-256-GCM (Advanced Encryption Standard, 256-bit key, Galois/Counter Mode) được sử dụng nhờ tính an toàn, hiệu suất cao và khả năng xác thực toàn vẹn dữ liệu tích hợp [9]. Để tăng cường khả năng chống tấn công brute-force, khóa mã hóa được sinh từ mật khẩu chính thông qua hàm dẫn xuất khóa PBKDF2 (Password-Based Key Derivation Function 2).

- Hệ thống sẽ triển khai PBKDF2 theo khuyến nghị của tiêu chuẩn NIST SP 800-132 [11]. Cơ chế này áp dụng một hàm giả ngẫu nhiên (PRF) như HMAC-SHA256 kết hợp với một chuỗi muối (salt) ngẫu nhiên, thực hiện lặp lại nhiều lần để tạo ra khóa dẫn xuất an toàn.

- Công thức dẫn xuất khóa được thực thi dựa trên đặc tả kỹ thuật của RFC 8018 [7]:

$$DK = \text{PBKDF2}(PRF, \text{Password}, \text{Salt}, c, dkLen) \quad (2.1)$$

Trong đó:

- *DK*: Khóa dẫn xuất (Derived Key).
- *PRF*: Hàm giả ngẫu nhiên (Pseudorandom Function).
- *Password*: Mật khẩu gốc của người dùng.
- *Salt*: Chuỗi dữ liệu ngẫu nhiên chống tấn công Rainbow Table.
- *c*: Số lần lặp (Iteration count).
- *dkLen*: Độ dài khóa dẫn xuất mong muốn.

2.1.2 Kiến trúc mã hóa phía máy khách (Client-side Encryption)

Kiến trúc mã hóa phía máy khách là mô hình thiết kế hệ thống trong đó toàn bộ quá trình mã hóa và giải mã được thực hiện tại thiết bị người dùng đảm bảo nhà cung cấp dịch vụ không có khả năng truy cập dữ liệu gốc. Mô hình này được xây dựng trên ba nguyên tắc cốt lõi [2] :

- Dữ liệu được mã hóa tại thiết bị người dùng trước khi truyền đi.
- Khóa giải mã không bao giờ rời khỏi thiết bị người dùng.
- Nền tảng lưu trữ chỉ tiếp nhận bản mã và không có công cụ để giải mã chúng .

Kiến trúc này loại bỏ rủi ro "điểm yếu tập trung" — ngay cả khi hạ tầng lưu trữ bị xâm phạm, dữ liệu gốc vẫn được bảo vệ nhờ tính chất toán học của thuật toán mã hóa. Dữ liệu được cache cục bộ phía trình duyệt luôn ở dạng ciphertext, trong khi khóa mã hóa chỉ tồn tại trong bộ nhớ tạm của phiên làm việc hiện tại.

2.1.3 Cơ chế định danh bằng google

Federated Authentication là mô hình xác thực trong đó ứng dụng ủy quyền việc xác minh danh tính cho một nhà cung cấp danh tính (Identity Provider – IdP) bên thứ ba thay vì tự quản lý thông tin đăng nhập. Chuẩn OAuth 2.0 (RFC 6749) [5] cung cấp nền tảng giao thức cho mô hình này. Thay vì người dùng phổ thông khi chưa quen phải tạo ví và mất thời gian tìm hiểu thì hệ thống áp dụng mô hình ví tất định - Deterministic Wallet [16]. Cụ thể sau khi xác thực thành công qua Firebase Authentication, hệ thống nhận một mã định danh duy nhất. Mã này sẽ kết hợp với chuỗi salt bí mật để tạo thành một khóa riêng tư và khóa riêng tư sẽ khởi tạo một ví ảo ngay trên RAM, điều này giúp tài khoản của người dùng có khả năng tự động kí các giao dịch

tương tác với smart contract - blockchain. Điều này giúp cải thiện và giảm thời gian phải tìm hiểu mà vẫn có thể sử dụng được ứng dụng. Cơ chế này chỉ đảm bảo việc hỗ trợ xác thực danh tính nhưng không tham gia vào quá trình mã hóa dữ liệu.

2.1.4 Lưu trữ kết hợp với IPFS và xác thực bằng blockchain

Để giải quyết bài toán chi phí và giới hạn không gian lưu trữ của blockchain hệ thống áp dụng kiến trúc lưu trữ kết hợp

- **Lưu trữ phân tán IPFS:** Dữ liệu mật khẩu sau khi được mã hóa AES-256 phía máy khách sẽ được đóng gói thành tệp JSON và tải lên mạng lưới InterPlanetary File System (IPFS). IPFS trả về một mã định danh nội dung duy nhất CID - Content Identifier dựa trên hàm băm mật mã học của chính tệp tin đó [1].
- **Xác thực toàn vẹn bằng Blockchain:** Blockchain là hệ thống sổ cái phân tán với đặc tính bất biến và minh bạch, trong đó dữ liệu được lưu trữ trên nhiều nút mạng thay vì một máy chủ trung tâm [8]. Trong dự án này mã CID sẽ được ví ảo ký xác nhận và ghi lên Smart Contract trên mạng Sepolia Testnet. Nhờ đặc tính bất biến của Blockchain [8], CID hoạt động như một thẻ định danh chứng minh tính toàn vẹn. Bất kỳ sự can thiệp trái phép nào vào dữ liệu trên IPFS sẽ làm thay đổi mã CID, khiến dữ liệu không còn khớp với bản ghi gốc trên Blockchain.

2.1.5 Công cụ phân tích dựa trên mô hình ước lượng entropy thực nghiệm

Việc người dùng sử dụng mật khẩu yếu là nguyên nhân phổ biến dẫn đến tấn công mạng [12]. Công cụ đánh giá độ mạnh mật khẩu dựa trên các thuật toán phân tích mẫu và ước lượng entropy (zxcvbn) giúp xác định mức độ an toàn của mật khẩu theo thời gian bẻ khóa ước tính [14].

2.2 Công nghệ sử dụng

2.2.1 Nền tảng Blockchain

Hệ thống sử dụng hạ tầng mạng lưới Ethereum [3] nhằm đảm bảo tính toàn vẹn dữ liệu thông qua hợp đồng thông minh:

- **Ngôn ngữ:** Solidity - ngôn ngữ lập trình hướng đối tượng chuyên dụng cho việc xây dựng hợp đồng thông minh.
- **Mạng thử nghiệm:** Sepolia Testnet - môi trường mô phỏng mạng chính thức để kiểm thử hiệu năng và bảo mật.

- **Công cụ phát triển:** Hardhat hoặc Remix IDE giúp quản lý vòng đời phát triển của mã nguồn.
- **Ví kết nối:** MetaMask đóng vai trò định danh và ký xác nhận các giao dịch trên mạng lưới.

Smart contract được thiết kế để lưu trữ hash SHA-256 của vault đã mã hóa kèm timestamp, phục vụ mục đích xác thực tính toàn vẹn dữ liệu thay vì lưu trữ ciphertext trực tiếp.

2.2.2 Frontend

Để đáp ứng yêu cầu xử lý phía máy khách (Client-side), hệ thống sử dụng bộ công cụ:

- **React.js:** Thư viện JavaScript hỗ trợ xây dựng giao diện người dùng theo kiến trúc thành phần.
- **Tailwind CSS:** Framework CSS tối ưu hóa quy trình thiết kế giao diện linh hoạt.
- **Ethers.js:** Thư viện trung gian kết nối ứng dụng với các nút mạng (nodes) của Blockchain.
- **item Firebase Authentication SDK:** Thư viện xác thực danh tính qua Google Sign-In, cung cấp UID dùng làm định danh người dùng [4].
- **Web Crypto API [13]:** API tích hợp sẵn trong trình duyệt, dùng để thực thi AES-256-GCM và PBKDF2 phía máy khách.

2.2.3 Mạng lưới lưu trữ phân tán IPFS

Hệ thống sử dụng IPFS (InterPlanetary File System) làm giải pháp lưu trữ dữ liệu chính nhằm khắc phục nút thắt cổ chai về chi phí (Gas fee) và không gian lưu trữ giới hạn của mạng lưới blockchain.

- **Giao thức mạng ngang hàng (P2P):** IPFS là một giao thức phân tán sử dụng cơ chế định danh dựa trên nội dung (Content-based addressing) thay vì định vị dựa trên vị trí máy chủ (Location-based addressing) như HTTP truyền thống [1].
- **Định danh nội dung (Content Identifier - CID):** Mỗi gói dữ liệu chứa mật khẩu đã mã hóa (Vault Payload) khi tải lên mạng lưới sẽ được băm mật mã học để sinh ra một chuỗi CID duy nhất. Bất kỳ sự thay đổi dù là nhỏ nhất trên gói dữ liệu sẽ dẫn đến một CID hoàn toàn khác, cung cấp cơ chế tự xác minh tính toàn vẹn mà không cần máy chủ trung gian.
- **Dịch vụ trung gian (IPFS Pinning Service):** Để tối ưu hóa tốc độ tải lên và đảm bảo tính khả dụng cao (High Availability) cho dữ liệu trên trình duyệt, hệ thống tích hợp API của các dịch vụ ghim dữ liệu chuyên dụng (như Pinata hoặc Infura). Các dịch vụ này đóng

vai trò như các nút (nodes) thường trực, giúp duy trì gói dữ liệu mã hóa luôn tồn tại trên mạng lưới IPFS toàn cầu.

2.2.4 Công nghệ lưu trữ cục bộ IndexedDB

Trong kiến trúc ứng dụng Web hiện đại và các ứng dụng phi tập trung (DApp) theo xu hướng ưu tiên ngoại tuyến (Local-First), IndexedDB đóng vai trò là cơ sở dữ liệu phi quan hệ (NoSQL) hiệu năng cao được tích hợp sẵn trong trình duyệt [10]. Thay vì bị giới hạn nghiêm ngặt về dung lượng (dưới 5MB) và chỉ hỗ trợ định dạng chuỗi đơn giản như `localStorage`, IndexedDB cho phép lưu trữ khối lượng lớn dữ liệu cấu trúc phức tạp như đối tượng JSON, tệp tin hoặc mảng nhị phân [15].

Cơ chế hoạt động của IndexedDB dựa trên mô hình giao dịch (Transactional Model), tự động hoàn tác (Rollback) nếu xảy ra lỗi nhằm bảo vệ tính toàn vẹn dữ liệu [15]. Đồng thời, việc thực thi truy vấn theo phương thức bất đồng bộ giúp ngăn chặn hiện tượng nghẽn luồng xử lý giao diện (UI Blocking), tối ưu hóa trải nghiệm người dùng kể cả khi xử lý các tập tin bản mã (Ciphertext) kích thước lớn. Đối với hệ thống quản lý mật khẩu an toàn, IndexedDB đóng vai trò cốt lõi tại tầng máy khách để lưu trữ bộ đệm kho dữ liệu đã mã hóa, giúp ứng dụng truy xuất tức thời và hoạt động độc lập mà không bị phụ thuộc vào độ trễ của mạng lưới Blockchain hay IPFS.

2.2.5 Công cụ đánh giá mật khẩu

Hệ thống sẽ tích hợp thư viện **zxcvbn-ts** dựa trên nghiên cứu về ước lượng entropy thực tế của mật khẩu [14]. Công cụ này cho phép phân tích độ mạnh mật khẩu dựa trên các mẫu (patterns) phổ biến mà không cần gửi dữ liệu về máy chủ, đảm bảo tính riêng tư tuyệt đối cho người dùng.

Chương 3

Phân tích và thiết kế hệ thống

Chương này trình bày toàn bộ phân tích yêu cầu cho hệ thống ứng dụng quản lý mật khẩu. Nội dung chương bao gồm: Đặc tả yêu cầu chức năng và phi chức năng.

3.1 Xác định các Tác nhân (Actors)

Dựa trên kiến trúc kỹ thuật của hệ thống Web3 DApp, các tác nhân tham gia tương tác vào hệ thống được phân chia thành hai nhóm chính bao gồm: tác nhân người dùng và các tác nhân hệ thống.

Tác nhân chính (Primary Actor)

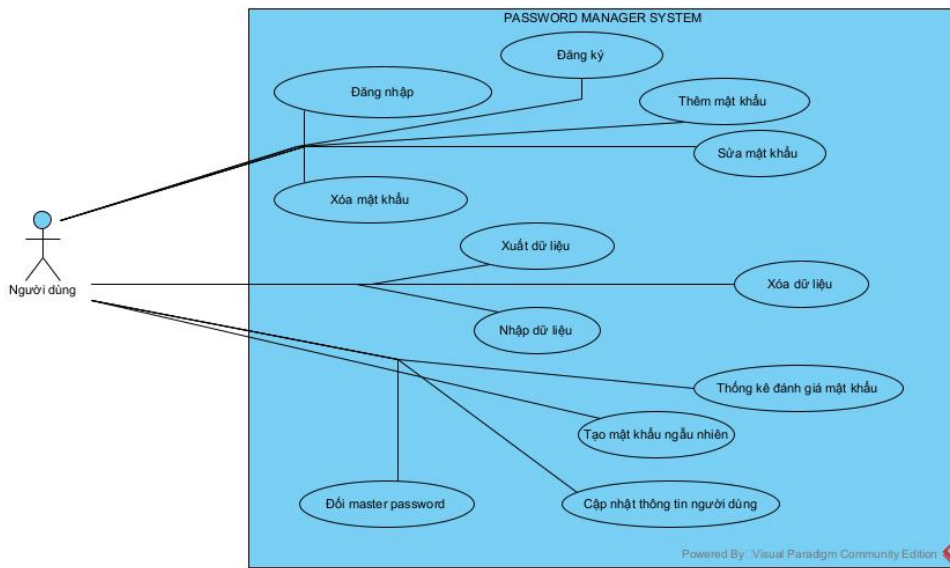
- **Người dùng (User / End-User):** Là đối tượng trung tâm sử dụng hệ thống để thực hiện các chức năng lưu trữ, quản lý thông tin tài khoản/mật khẩu cá nhân. Người dùng trực tiếp thực hiện các thao tác mã hóa dữ liệu cục bộ và ra lệnh đồng bộ dữ liệu lên môi trường phân tán.

Các tác nhân hệ thống (System Actors)

- **Nhà cung cấp định danh (Identity Provider - Firebase Google Auth):** Thành phần chịu trách nhiệm xác thực danh tính ban đầu của người dùng thông qua giao thức OAuth 2.0, cung cấp mã định danh duy nhất (*User ID - UID*) làm nền tảng đầu vào cho thuật toán băm sinh ví tất định cục bộ.
- **Ví Blockchain (MetaMask / Web3 Provider):** Tác nhân trung gian đóng vai trò quản lý khóa bí mật của tài khoản Web3, thực hiện ký xác thực các giao dịch và tương tác trực tiếp với mạng lưới Blockchain thông qua giao thức gọi hàm từ xa (*RPC - Remote Procedure Call*).

- **Mạng lưu trữ phi tập trung (IPFS Network):** Hệ thống lưu trữ các tệp tin chứa kho mật khẩu (*Vault*) đã được mã hóa an toàn dưới dạng các định danh nội dung duy nhất (*CID - Content Identifier*).
- **Mạng lưới Blockchain (Ethereum Sepolia Testnet):** Môi trường phân tán chịu trách nhiệm thực thi mã nguồn và lưu trữ vĩnh viễn trạng thái của các con trỏ địa chỉ IPFS thông qua hợp đồng thông minh (*Smart Contract*) có tên VaultPointer.

3.2 Sơ đồ usecase Tổng quát



Hình 3.1: Sơ đồ usecase tổng quát

3.3 Chi tiết usecase

3.3.1 Tổng hợp danh sách các ca sử dụng (Use Cases)

Hệ thống Quản lý Mật khẩu được phân rã thành 12 usecase sử dụng chính bao gồm:

1. **UC1:** Đăng nhập
2. **UC2:** Đăng ký
3. **UC3:** Thêm mật khẩu mới
4. **UC4:** Sửa thông tin mật khẩu
5. **UC5:** Xóa mật khẩu
6. **UC6:** Tạo mật khẩu mạnh ngẫu nhiên

7. **UC7:** Xuất dữ liệu (Export JSON)
8. **UC8:** Nhập dữ liệu (Import JSON)
9. **UC9:** Xóa toàn bộ dữ liệu kho lưu trữ
10. **UC10:** Báo cáo bảo mật (Phân tích rủi ro, trùng lặp)
11. **UC11:** Đổi mật khẩu chủ (Master Password)
12. **UC12:** Cập nhật thông tin cá nhân

3.3.2 UC1: Đăng nhập

Thuộc tính	Đặc tả chi tiết
Tên Usecase	Đăng nhập
Tác nhân (Actors)	Người dùng
Tiền điều kiện	Người dùng đã truy cập ứng dụng và đã đăng kí tài khoản trên hệ thống
Hậu điều kiện	Phiên đăng nhập được thiết lập
Luồng sự kiện chính	1. Người dùng nhấn vào chức năng đăng nhập 2. Hệ thống hiển thị giao diện đăng nhập: • Ô nhập master password • Nút đăng nhập bằng google • nút đăng nhập bằng metamask 3. Người dùng Nhấn vào đăng nhập bằng google 4. Hệ thống kết nối và hiển thị popup đăng nhập tài khoản 5. Người dùng nhập thông tin và đăng nhập 6. Hệ thống quay về giao diện đăng nhập và yêu cầu nhập master password 7. Người dùng nhập master password: 123456674 và ấn đăng nhập bằng google lần nữa 8. Hệ thống xác thực và chuyển người dùng đến trang chủ.

Thuộc tính	Đặc tả chi tiết (Tiếp tục)
Luồng rẽ nhánh & Ngoại lệ	• 3.1: Người dùng nhập master password:24034939 • 3.2: Hệ thống mở nút đăng nhập bằng metamask • 3.3: Người dùng ấn vào đăng nhập bằng metamask • 3.4: Hệ thống kết nối với extension metamask trên trình duyệt • 3.5: Người dùng cho phép connect • 3.6 : Chuyển đến bước 8 của luồng chính • 5.1 Người dùng bấm tắt popup • 5.2 Hệ thống báo lỗi và quay về bước 2 • 6.1 Hệ thống báo tài khoản google chưa được đăng kí • 6.2 Hệ thống chuyển sang giao diện đăng kí • 8.1 Hệ thống báo sai masterpassword

3.3.3 UC2: Đăng ký

Thuộc tính	Đặc tả chi tiết
Tên Usecase	Đăng nhập bằng ví metamask
Tác nhân (Actors)	Người dùng
Tiền điều kiện	Người dùng đã truy cập ứng dụng và đã đăng kí tài khoản trên hệ thống
Hậu điều kiện	Phiên đăng nhập được thiết lập

Thuộc tính	Đặc tả chi tiết (Tiếp tục)
Luồng sự kiện chính	1. Người dùng nhấn vào chức năng đăng ký 2. Hệ thống hiển thị giao diện đăng ký: • Ô tạo master password • Ô xác nhận master password • Tùy chọn Đăng kí thông tin cá nhân • Nút đăng ký bằng google • nút đăng ký bằng metamask 3. Người dùng Nhấn vào đăng ký bằng google 4. Hệ thống kết nối và hiển thị popup đăng nhập tài khoản 5. Người dùng nhập thông tin và đăng nhập 6. Hệ thống quay về giao diện đăng ký và yêu cầu tạo master password 7. Người dùng nhập master password: 123456674 và xác nhận master password : 123456674 8. Hệ thống hiển thị đánh giá mật khẩu 9. Người dùng nhấn đăng ký bằng google lần nữa 10. Hệ thống xác thực và chuyển người dùng đến trang chủ. • 3.1 Người dùng nhập Master password và xác nhận master password • 3.2 Hệ thống mở nút đăng ký bằng metamask • 3.3 Người dùng nhấn vào nút đăng ký bằng metamask • 3.4 Hệ thống kết nối với extension metamask trên trình duyệt • 3.5: Người dùng cho phép connect • 3.6: Chuyển đến bước 10 • 7.1 Người dùng nhấn tùy chọn nhập thông tin cá nhân • 7.2 Hệ thống hiển thị: – Ô nhập Họ và tên – Ô nhập ngày tháng năm sinh – Ô nhập Số điện thoại • 7.3 Người dùng nhập các thông tin muốn cung cấp Họ và tên: Hoàng, ... • 7.4 Tiếp tục ở bước 7 trong luồng chính • 8.1 Hệ thống đánh giá mật khẩu yếu • 8.2 Người dùng thực hiện lại bước 7 • 5.1 Người dùng bấm tắt popup • 5.2 Hệ thống báo lỗi và quay về bước 2
Luồng rẽ nhánh & Ngoại lệ	

3.3.4 UC3: Thêm mật khẩu

Thuộc tính	Đặc tả chi tiết
Tên Usecase	Thêm mật khẩu
Tác nhân (Actors)	Người dùng
Tiền điều kiện	Người dùng đăng nhập thành công
Hậu điều kiện	Mật khẩu được xử lý mã hóa và lưu thành công
Luồng sự kiện chính	1. Người dùng ấn vào nút Thêm mật khẩu trong giao diện trang chủ 2. Hệ thống hiển thị popup thêm mật khẩu mới: • Ô nhập url • Ô nhập username • Ô nhập mật khẩu • Nút Lưu 3. Người dùng Nhập dữ liệu • url : miniminshop.com • username : jjin • Mật khẩu: 123 4. Hệ thống hiển thị đánh giá mật khẩu 5. Người dùng nhấn lưu 6. Hệ thống hiển thị giao diện thông báo chờ mã hóa, lưu dữ liệu và thông báo hoàn tất sau khi thực hiện xong • 6.1 Lỗi lưu mật khẩu hệ thống sẽ hiển thị thông báo lưu thất bại
Luồng rẽ nhánh & Ngoại lệ	

3.3.5 UC4: Sửa mật khẩu

Thuộc tính	Đặc tả chi tiết
Tên Usecase	Sửa mật khẩu
Tác nhân (Actors)	Người dùng
Tiền điều kiện	Người dùng đăng nhập thành công và có ít nhất 1 mật khẩu đã lưu
Hậu điều kiện	Mật khẩu được xử lý mã hóa và thay đổi thành công

Thuộc tính	Đặc tả chi tiết (Tiếp tục)
Luồng sự kiện chính	1. Người dùng ấn vào biểu tượng sửa dòng mật khẩu muốn sửa 2. Hệ thống hiển thị popup chỉnh sửa mật khẩu: • Ô nhập url • Ô nhập username • Ô nhập mật khẩu • Nút Lưu 3. Người dùng Nhập dữ liệu • url : miniminshop.com • username : jjin • Mật khẩu: 123 4. Hệ thống hiển thị đánh giá mật khẩu 5. Người dùng nhấn lưu 6. Hệ thống hiển thị giao diện thông báo chờ mã hóa, lưu dữ liệu và thông báo hoàn tất sau khi thực hiện xong
Luồng rẽ nhánh & Ngoại lệ	• 6.1 Lỗi lưu mật khẩu hệ thống sẽ hiển thị thông báo lưu thất bại

3.3.6 UC5: Xóa mật khẩu

Thuộc tính	Đặc tả chi tiết
Tên Usecase	Xóa mật khẩu
Tác nhân (Actors)	Người dùng
Tiền điều kiện	Người dùng đăng nhập thành công và có ít nhất 1 mật khẩu đã lưu
Hậu điều kiện	Mật khẩu được xử lý mã hóa và thay đổi thành công
Luồng sự kiện chính	1. Người dùng ấn vào biểu tượng xóa dòng mật khẩu muốn xóa 2. Hệ thống hiển thị giao diện thông báo chờ thông báo hoàn tất sau khi thực hiện xong
Luồng rẽ nhánh & Ngoại lệ	• 6.1 Lỗi xóa mật khẩu hệ thống sẽ hiển thị thông báo thất bại

3.3.7 UC6: Tạo mật khẩu mạnh ngẫu nhiên

Thuộc tính	Đặc tả chi tiết
Tên Usecase	Tạo mật khẩu mạnh ngẫu nhiên
Tác nhân (Actors)	Người dùng
Tiền điều kiện	Người dùng đăng nhập thành công
Hậu điều kiện	Mật khẩu được tạo cho người dùng sao chép và dùng
Luồng sự kiện chính	1. Người dùng ấn Trình tạo ở menu 2. Hệ thống hiển thị giao diện tạo • Thanh tùy chọn độ dài mong muốn • 3 Ô tích Chữ hoa / Số / Ký tự đặc biệt 3. Người dùng chọn và nhấn tạo mật khẩu mới 4. Mật khẩu mới ngẫu nhiên được sinh ra 5. Người dùng sao chép và sử dụng

3.3.8 UC7: Xuất dữ liệu

Thuộc tính	Đặc tả chi tiết
Tên Usecase	Xuất dữ liệu
Tác nhân (Actors)	Người dùng
Tiền điều kiện	Người dùng đăng nhập thành công
Hậu điều kiện	Xuất file json thành công
Luồng sự kiện chính	1. Người dùng ấn nút xuất file JSON mã hóa trong phần Nhập & xuất 2. Hệ thống hiển thị yêu cầu nhập master password 3. Người dùng nhập 4. Hệ thống bảo lưu thành công • 3.1 Người dùng nhập sai • 3.2 Hệ thống báo sai và yêu cầu nhập lại
Luồng rẽ nhánh & Ngoại lệ	

3.3.9 UC8: Nhập dữ liệu

Thuộc tính	Đặc tả chi tiết
Tên Usecase	Nhập dữ liệu
Tác nhân (Actors)	Người dùng

Thuộc tính	Đặc tả chi tiết (Tiếp tục)
Tiền điều kiện	Người dùng đăng nhập thành công và có file lưu mật khẩu theo đúng định dạng
Hậu điều kiện	Toàn bộ mật khẩu trong file nhập được thêm thành công
Luồng sự kiện chính	1. Người dùng ấn nút nhập file JSON mã hóa trong phần Nhập & xuất và chọn file cần nhập 2. Hệ thống hiển thị xác nhận: • Ô nhập master password (nếu có) • Hủy • bỏ qua (File dữ liệu là bản rõ không cần master password) • Giải mã & nhập (File dữ liệu theo chuẩn hệ thống) 3. Người dùng nhập master nhấn Giải mã & Nhập 4. Hệ thống báo chờ và hiển thị phân tích mật khẩu trong file nhập sau khi hoàn tất

3.3.10 UC9: Xóa dữ liệu

Thuộc tính	Đặc tả chi tiết
Tên Usecase	Xóa dữ liệu
Tác nhân (Actors)	Người dùng
Tiền điều kiện	Người dùng đăng nhập thành công
Hậu điều kiện	Toàn bộ mật khẩu đang lưu được xóa thành công
Luồng sự kiện chính	1. Người dùng ấn Xóa toàn bộ dữ liệu trong phần Nhập & xuất và chọn file cần nhập 2. Hệ thống hiển thị xác nhận: • Ô nhập master password (nếu có) 3. Người dùng nhập master và nhấn xác nhận 4. Hệ thống báo chờ và hiển thị xóa thành công

3.3.11 UC10: Báo cáo bảo mật

Thuộc tính	Đặc tả chi tiết
Tên Usecase	Báo cáo bảo mật

Thuộc tính	Đặc tả chi tiết (Tiếp tục)
Tác nhân (Actors)	Người dùng
Tiền điều kiện	Người dùng đăng nhập thành công
Hậu điều kiện	Người dùng nhận được báo cáo về mật khẩu đang được lưu
Luồng sự kiện chính	1. Người dùng ấn Báo trong phần menu 2. Hệ thống xử lý và hiển thị giao diện bao gồm: • Tổng số tài khoản • Mật khẩu yếu • Trùng lặp • Vi phạm thông tin • Bảng phân tích chi tiết tính rủi ro của từng mật khẩu

3.3.12 UC11: Đổi master password

Thuộc tính	Đặc tả chi tiết
Tên Usecase	Đổi master password
Tác nhân (Actors)	Người dùng
Tiền điều kiện	Người dùng đăng nhập thành công
Hậu điều kiện	Master được đổi thành công
Luồng sự kiện chính	1. Người dùng nhấn vào Cài đặt bảo mật ở menu 2. Hệ thống hiển thị phần Đổi Master password: • Ô nhập mật khẩu hiện tại • Ô nhập mật khẩu mới • Ô xác nhận • Nút cập nhật 3. Người dùng nhập 4. Hệ thống hiển thị chờ và báo thành công khi hoàn tất

3.3.13 UC12: Cập nhật Thông tin cá nhân

Thuộc tính	Đặc tả chi tiết
Tên Usecase	Cập nhật Thông tin cá nhân
Tác nhân (Actors)	Người dùng

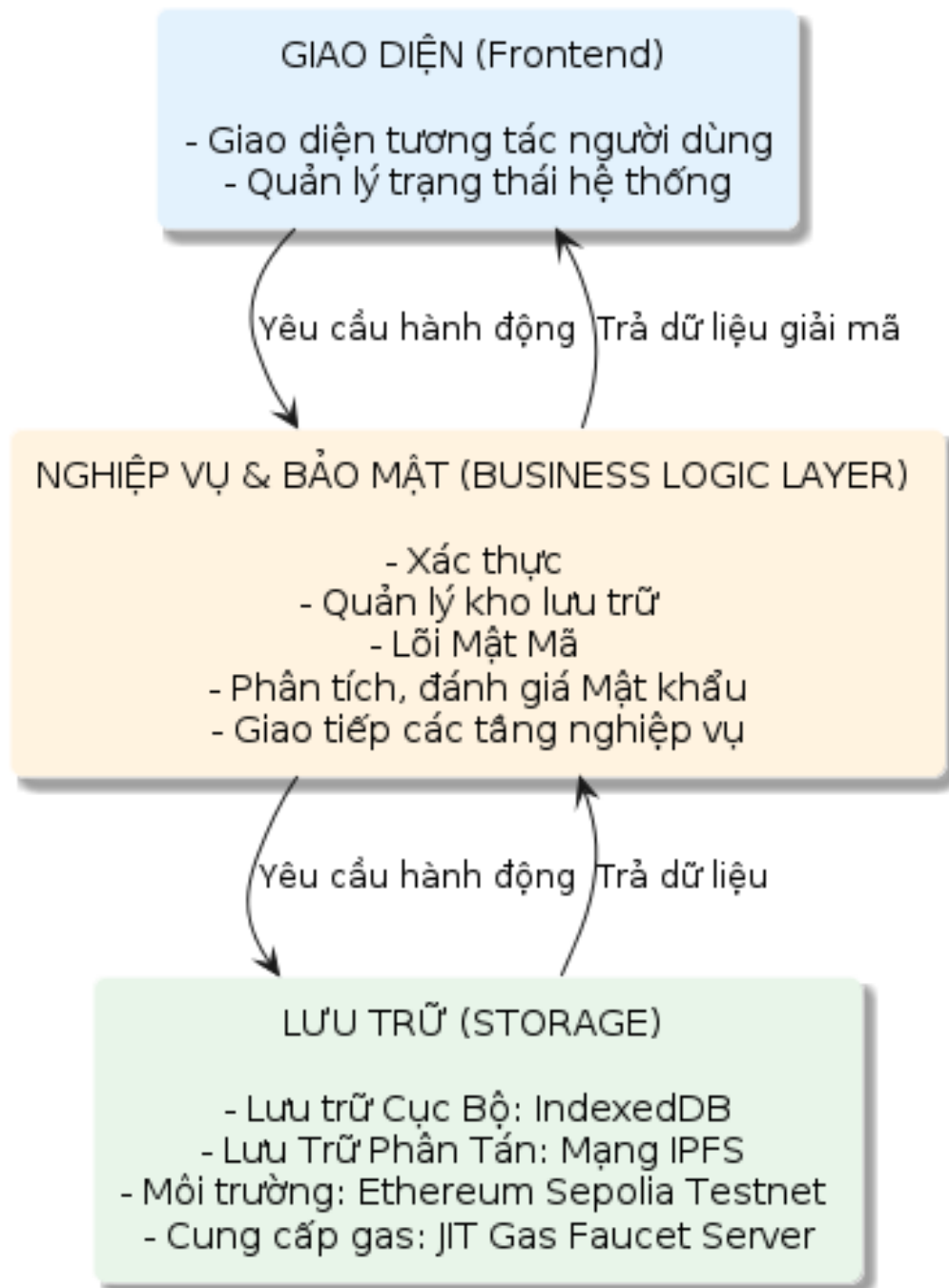
Thuộc tính	Đặc tả chi tiết (Tiếp tục)
Tiền điều kiện	Người dùng đăng nhập thành công
Hậu điều kiện	Thông tin cá nhân được cập nhật thành công
Luồng sự kiện chính	1. Người dùng nhấn phần Hồ sơ cá nhân trong Cài đặt bảo mật ở menu 2. Hệ thống hiển thị phần Cập nhật thông tin: • Ô nhập Họ và tên • Ô nhập ngày tháng năm sinh • Ô nhập số điện thoại • Nút đồng bộ 3. Người dùng nhập thông tin muốn thay đổi 4. Hệ thống hiển thị chờ và báo thành công khi hoàn tất

3.4 Thiết kế kiến trúc hệ thống

- **Tầng Giao diện (Frontend / Presentation Layer):** Đóng vai trò là điểm tiếp xúc trực tiếp với người dùng. Tầng này tuân thủ nguyên tắc an ninh tối giản, không lưu trữ bất kỳ dữ liệu nhạy cảm hay khóa mật mã cố định nào trên bộ nhớ vật lý. Nhiệm vụ chính của nó là thu thập thông tin đầu vào, quản lý trạng thái phiên làm việc hiện tại trên bộ nhớ tạm (RAM), hiển thị các cảnh báo tương tác (Toast/Alert) và trình bày dữ liệu dạng bản rõ (Plaintext) sau khi nhận được kết quả giải mã an toàn từ tầng nghiệp vụ.
- **Tầng Nghiệp vụ và Bảo mật (Business Logic Layer):** Đây là trung tâm xử lý cốt lõi, đóng vai trò chính của toàn bộ hệ thống. Tầng này tích hợp cơ chế xác thực kép linh hoạt (kết hợp Firebase Google Auth cho người dùng Web2 và kết nối MetaMask trực tiếp cho người dùng Web3). Đồng thời, lớp nghiệp vụ chịu trách nhiệm phân tích độ mạnh mật khẩu cục bộ tránh lộ thông tin (sử dụng thư viện zxcvbn). Đặc biệt, lõi mật mã thực hiện các thuật toán mã hóa đầu-cuối (PBKDF2-SHA256 để dẫn xuất khóa và AES-256-GCM để bảo vệ dữ liệu) hoàn toàn ở phía máy khách. Điều này đảm bảo dữ liệu luôn ở dạng bản mã trước khi rời khỏi trình duyệt.
- **Tầng Lưu trữ Storage Layer):** Áp dụng kiến trúc lưu trữ lai (Hybrid Storage) đặc thù để giải quyết bài toán hiệu năng và chi phí giao dịch trên blockchain. Dữ liệu bản mã được đồng bộ đồng thời trên ba môi trường:
 - Bộ nhớ cục bộ (IndexedDB): Lưu trữ bộ đệm Ciphertext riêng biệt theo từng địa chỉ ví tại trình duyệt, cho phép truy xuất nhanh và hỗ trợ chế độ hoạt động ngoại tuyến.
 - Mạng lưu trữ phân tán (IPFS): Lưu trữ phi tập trung các gói tin định dạng JSON

chứa dữ liệu kết sắt đã được mã hóa thông qua Pinata API.

- Môi trường (Ethereum Sepolia Testnet): Ghi nhận các con trỏ định danh nội dung (CID) lên Smart Contract.
- Cung cấp gas (JIT Gas Faucet Server): Máy chủ hỗ trợ cấp phí gas tự động theo cơ chế JIT (Just-In-Time Gas Faucet) nhằm tối ưu hóa trải nghiệm ký giao dịch của người dùng không chuyên Web3.



Hình 3.2: Sơ đồ tổng quát kiến trúc phân tầng của hệ thống

3.4.1 Thiết kế chi tiết Tầng Lưu trữ (Storage Layer)

Tổng quan kiến trúc lưu trữ lai (Hybrid Storage - Web 2.5)

Hệ thống quản lý mật khẩu phân tán được thiết kế theo mô hình kiến trúc lai **Web 2.5**. Tầng lưu trữ của hệ thống giải quyết bài toán cốt lõi: đảm bảo tính bảo mật tuyệt đối của thông tin cá nhân (mô hình Zero-Knowledge đầu-cuối), đồng thời duy trì khả năng truy cập linh hoạt và tính toàn vẹn dữ liệu thông qua công nghệ Web3.

Dữ liệu trong hệ thống không tồn tại ở một trạng thái duy nhất mà được phân rã thành 4 lớp lưu trữ bổ trợ lẫn nhau, đi từ lớp bộ nhớ biến động (Volatile) đến lớp lưu trữ vĩnh viễn không thể thay đổi (Immutable):

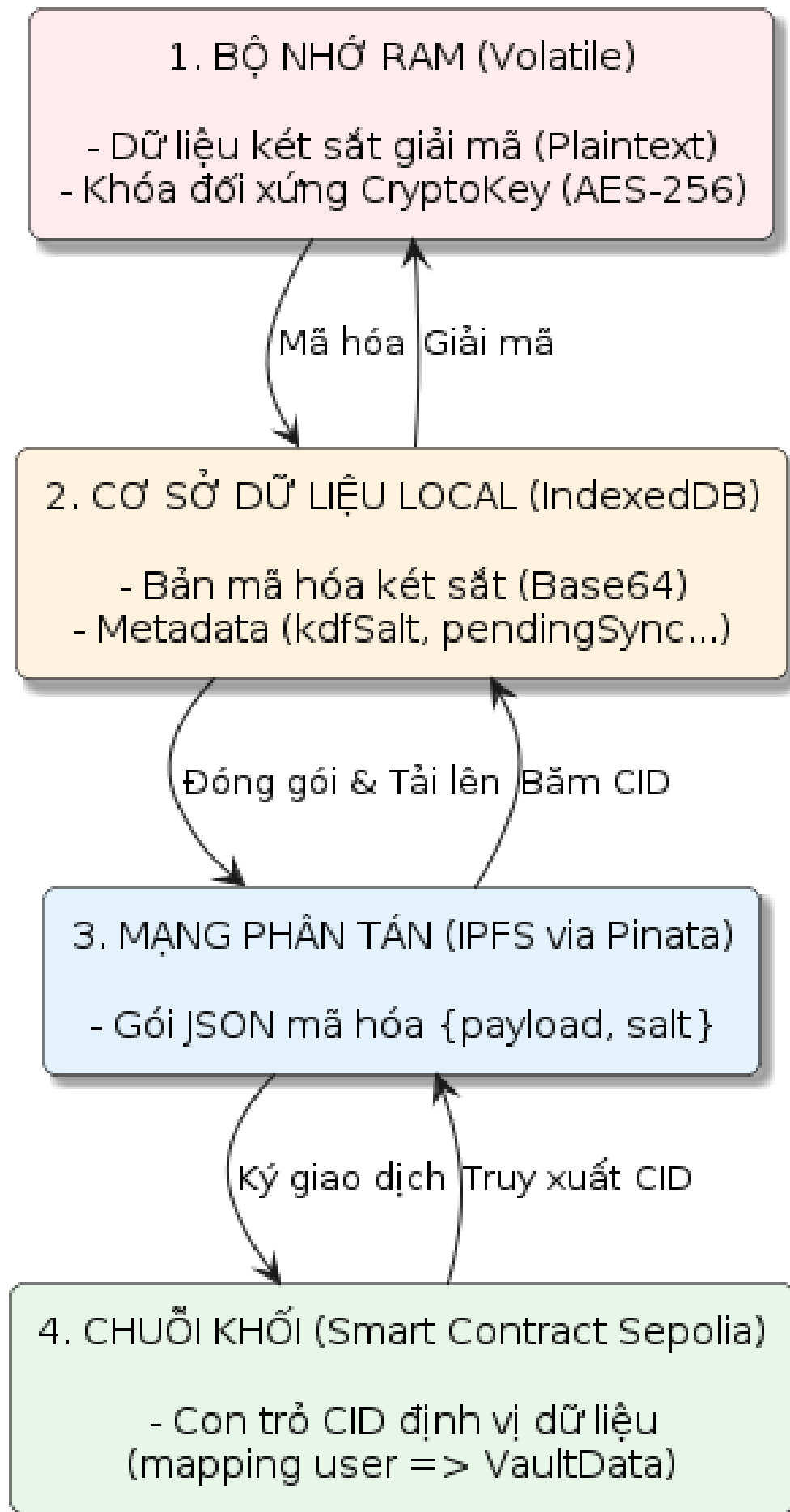
1. **Bộ nhớ RAM (Biến động - Volatile):** Là nơi duy nhất tồn tại dữ liệu kết sắt dưới dạng bản rõ (Plaintext) để hiển thị lên giao diện UI và lưu trữ khóa mật mã đối xứng (CryptoKey AES-256). Dữ liệu tại đây sẽ bị xóa sạch ngay khi kết thúc phiên làm việc.
2. **Cơ sở dữ liệu IndexedDB (Cục bộ - Local Cache):** Lưu trữ bản mã hóa của kết sắt (dưới dạng chuỗi Base64) và các siêu dữ liệu (*Metadata*) hỗ trợ quá trình đồng bộ như: muối sinh khóa (kdfSalt), mã băm CID đồng bộ lần cuối (lastSyncedCid) và cờ đánh dấu trạng thái chờ đồng bộ (pendingSync).
3. **Mạng lưu trữ IPFS (Phân tán - Decentralized):** Sử dụng dịch vụ Pinata để ghim (pin) các tệp tin lưu trữ. Gói dữ liệu đẩy lên IPFS là một cấu trúc JSON đã được mã hóa hoàn toàn, bao gồm encryptedPayload và salt.
4. **Smart Contract Sepolia (Chuỗi khối - On-chain):** Đóng vai trò là sổ cái lưu trữ trạng thái cuối cùng thông qua một cấu trúc ánh xạ đơn giản: `mapping(address => string)` nhằm liên kết địa chỉ ví của người dùng với con trỏ CID trên IPFS.

Cơ chế đồng bộ và Xoay vòng khóa (State Synchronization & Key Rotation)

Hệ thống thiết kế các luồng đồng bộ trạng thái độc lập để tối ưu hóa hiệu suất và bảo mật:

- **Luồng cập nhật cục bộ (Local-First):** Khi người dùng thêm, sửa hoặc xóa mật khẩu, hệ thống chỉ thực hiện mã hóa và ghi đè bản mã (Ciphertext) xuống IndexedDB cục bộ, đồng thời bật cờ `pendingSync = true`. Quá trình này diễn ra tức thời, không phụ thuộc vào độ trễ của mạng lưới Blockchain.
- **Luồng đồng bộ Cloud Web3:** Khi người dùng chủ động yêu cầu đồng bộ, hệ thống sẽ đóng gói toàn bộ dữ liệu từ IndexedDB, đẩy lên IPFS để nhận về mã băm CID mới. Sau đó, hệ thống gọi hàm trên Smart Contract để cập nhật con trỏ này lên mạng lưới Ethereum Sepolia.

- **Luồng xoay vòng khóa (Master Password Rotation):** Đây là tiến trình bảo mật đặc biệt quan trọng. Khi người dùng đổi mật khẩu chủ, hệ thống yêu cầu cả mật khẩu cũ và mới. Hệ thống sẽ dẫn xuất khóa giải mã cũ (`oldKey`) để giải mã toàn bộ kết nối về dạng bản rõ, sau đó lập tức sử dụng khóa mới (`nextKey`) để tái mã hóa toàn bộ dữ liệu. Bản mã mới này được lưu xuống IndexedDB và tự động kích hoạt tiến trình đồng bộ lên IPFS và Blockchain để ghi nhận trạng thái mới.



3.4.2 Thiết kế chi tiết Tầng Nghiệp vụ và Bảo mật (Business Logic & Security Layer)

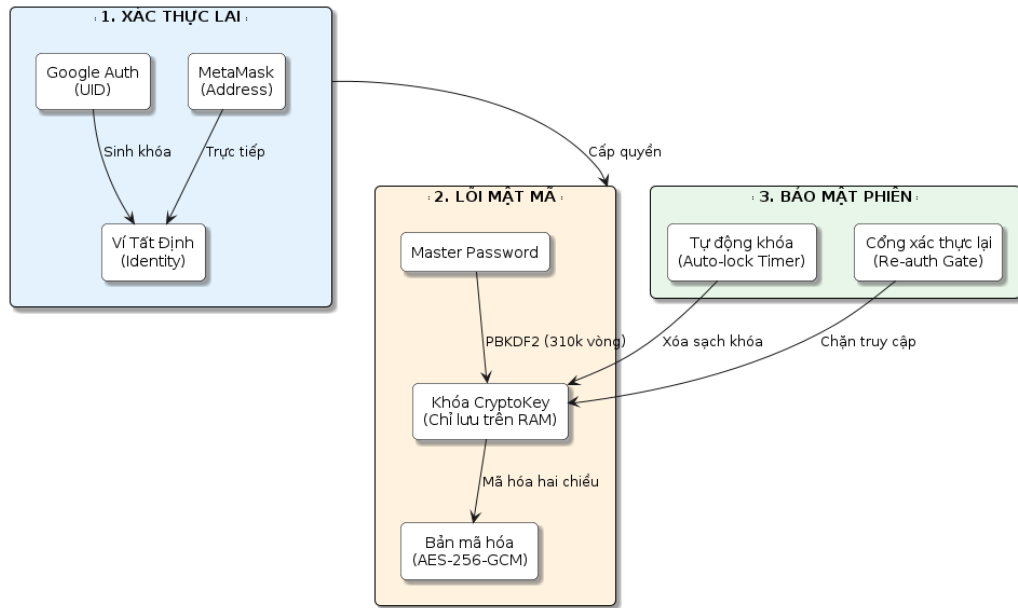
Tầng Nghiệp vụ và Bảo mật đóng vai trò trung tâm điều phối, tuân thủ tuyệt đối Kiến trúc mã hóa phía máy khách, đảm bảo máy chủ không bao giờ tiếp cận được dữ liệu bản rõ của người dùng. Tầng này bao gồm 4 module cốt lõi:

- **Xác thực kép lai (Hybrid Authentication):** Hỗ trợ đồng thời người dùng phổ thông qua Google Auth (băm UID thành ví tắt định) và người dùng Web3 bản địa thông qua tiện ích MetaMask.
- **Lõi mật mã (Client-Side Crypto Core):** Mọi thao tác mật mã học được thực thi trực tiếp tại trình duyệt.
 - *Dẫn xuất khóa:* Sử dụng thuật toán PBKDF2-SHA256 với độ trễ cố ý (310.000 vòng lặp) để sinh khóa CryptoKey từ mật khẩu chủ, giúp chống tấn công Brute-force.
 - *Mã hóa/Giải mã:* Sử dụng tiêu chuẩn AES-256-GCM chuẩn quân đội để xử lý dữ liệu.
- **Quản lý trạng thái phiên (Session State):** Áp dụng chính sách **RAM-Only**. Khóa giải mã chỉ tồn tại tạm thời trên RAM và bị hủy ngay lập tức bởi bộ đếm thời gian (Auto-lock) hoặc khi người dùng đăng xuất. Các thao tác nhạy cảm (xuất file, xem mật khẩu) đều phải đi qua Cổng xác thực lại (Re-auth Gate).
- **Đánh giá an toàn cục bộ (Password Intelligence):** Tích hợp thư viện zxcvbn để chấm điểm entropy của mật khẩu, kết hợp với từ điển động trích xuất từ thông tin cá nhân (tên, ngày sinh) để chặn người dùng đặt mật khẩu dễ đoán.

Đánh giá năng lực phòng thủ

Thiết kế kiến trúc mật mã của hệ thống cho phép phòng chống hiệu quả các rủi ro:

1. **Chống Sniffing:** Dữ liệu đẩy lên mạng (IPFS/Blockchain) luôn ở trạng thái Ciphertext, vô nghĩa với kẻ tấn công nghe lén.
2. **Chống rò rỉ khóa:** Khóa CryptoKey được cấu hình `extractable: false`, ngăn chặn các extension độc hại trên trình duyệt trích xuất khóa khỏi bộ nhớ.
3. **Chống Timing Attacks:** Cổng xác thực lại (Re-auth Gate) sử dụng thuật toán so sánh thời gian không đổi (Constant-Time) để đối chiếu mã băm.



Hình 3.4: Sơ đồ kiến trúc tầng nghiệp vụ, bảo mật

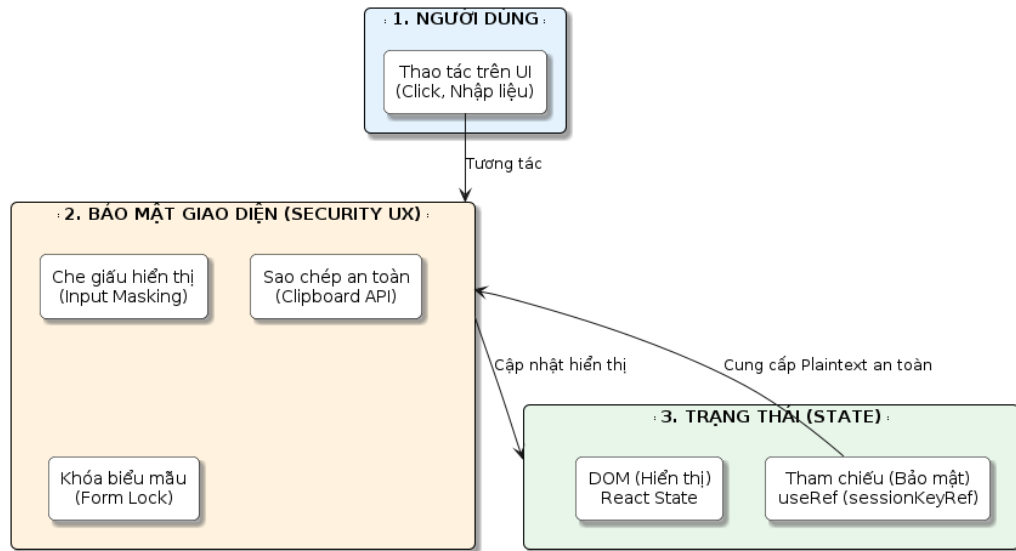
3.4.3 Thiết kế chi tiết Tầng Giao diện (Frontend / Presentation Layer)

Tầng Giao diện là điểm tiếp xúc trực tiếp duy nhất giữa người dùng và hệ thống, được xây dựng theo mô hình Ứng dụng Trang đơn (Single Page Application - SPA) dựa trên thư viện **ReactJS** và **Tailwind CSS**. Thiết kế của tầng này tập trung vào sự cân bằng giữa trải nghiệm người dùng (UX) và nguyên tắc an ninh tối giản (Zero-Persistence).

Các thành phần cốt lõi bao gồm:

- **Kiến trúc Component trực quan:** Phân tách rõ ràng các không gian tương tác như Không gian xác thực (AuthPage), Không gian quản lý kho (VaultPage, VaultPanel) và Không gian thiết lập bảo mật (SecurityPage).
- **Quản lý Trạng thái an toàn (State Management):** Sử dụng React Context API kết hợp với useRef. Khóa mật mã CryptoKey được lưu giữ trong tham chiếu đột biến (*Mutable Ref*) thay vì State thông thường, giúp hệ thống không bị render lại (re-render) không cần thiết, đồng thời ngăn chặn nguy cơ rò rỉ khóa ra ngoài giao diện.
- **Cơ chế Bảo mật Giao diện (Security UX):**
 - *Che giấu dữ liệu (Input Masking):* Các trường mật khẩu dạng bản rõ (Plaintext) luôn được che giấu mặc định.
 - *Sao chép an toàn (Clipboard API):* Hỗ trợ người dùng sao chép mật khẩu trực tiếp vào bộ nhớ đệm (Clipboard) của hệ điều hành mà không cần hiển thị văn bản ra màn hình, chống lại các rủi ro quay lén hoặc nhìn trộm (Shoulder Surfing).

- *Khóa tương tác (Form Lock)*: Tự động vô hiệu hóa các biểu mẫu trong quá trình chờ mạng lưới Web3 xác nhận giao dịch, ngăn chặn thao tác gửi trùng lặp gây lãng phí phí Gas.



Hình 3.5: Sơ đồ kiến trúc tầng giao diện

3.4.4 Thiết kế hệ thống giao diện lập trình ứng dụng (API Design)

Hệ thống quản lý mật khẩu phi tập trung vận hành dựa trên tập hợp các giao diện lập trình ứng dụng (API) nội bộ và đối ngoại được thiết kế đồng bộ hoàn toàn với sơ đồ lớp kiến trúc (Class Diagram). Việc phân rã API giúp đảm bảo tính đóng gói dữ liệu và phân tách trách nhiệm xử lý rõ ràng giữa các phân tầng Presentation - Business - Storage.

3.5 Đặc tả chi tiết danh mục chức năng API (API Directory)

Hệ thống quản lý mật khẩu phi tập trung vận hành dựa trên tập hợp các giao diện lập trình ứng dụng (API) nội bộ và đối ngoại được phân rã tường minh. Việc chuyển đổi cấu trúc sang danh mục đặc tả chi tiết dưới đây giúp làm rõ tham số đầu vào, đầu ra và nhiệm vụ xử lý cốt lõi của từng dịch vụ.

3.5.1 Danh mục chức năng API theo các phân tầng kiến trúc

Tầng Giao diện và Quản lý Trạng thái (Presentation Layer API)

Dịch vụ quản lý trạng thái toàn cục `AppContext.jsx` chịu trách nhiệm đóng gói các biến trạng thái runtime, cung cấp các hành động hướng sự kiện cho tầng giao diện:

`signInGoogle()`

Đầu vào: Không có (none)

Đầu ra: `Promise<void>`

Nhiệm vụ: Ủy quyền và khởi chạy tiến trình đăng nhập bằng Google cho tầng giao diện thông qua Firebase OAuth2.

`connectWallet()`

Đầu vào: Không có (none)

Đầu ra: `Promise<void>`

Nhiệm vụ: Gửi yêu cầu kết nối với ví điện tử MetaMask và trích xuất địa chỉ ví (*public address*) của người dùng.

`runSessionUnlock(password)`

Đầu vào: `password: String` (Mật khẩu chủ Master Password)

Đầu ra: `Promise<void>`

Nhiệm vụ: Xác thực tính hợp lệ của mật khẩu chủ, kích hoạt tiến trình dẫn xuất khóa mật mã và mở khóa phiên làm việc trên bộ nhớ tạm.

`lockSession()`

Đầu vào: Không có (none)

Đầu ra: `void`

Nhiệm vụ: Thu hồi phiên làm việc ngay lập tức, khóa giao diện và xóa sạch khóa đối xứng khỏi bộ nhớ RAM để đảm bảo an toàn.

`setVaults(nextVaults, options)`

Đầu vào: `nextVaults: Array<VaultItem>`, `options: Object`

Đầu ra: `Promise<void>`

Nhiệm vụ: Tiếp nhận mảng dữ liệu rõ mới, điều phối tiến trình mã hóa, sao lưu phi tập trung và đồng bộ lên hạ tầng Web3.

`updateMasterPassword(current, next)`

Đầu vào: `current: String` (Mật khẩu cũ), `next: String` (Mật khẩu mới)

Đầu ra: `Promise<void>`

Nhiệm vụ: Thực thi quy trình giải mã kho khóa cũ bằng khóa cũ, dẫn xuất hệ thống khóa mới và mã hóa lại toàn bộ kết sắt (*Key Rotation*).

Tầng Nghiệp vụ và Bảo mật (Business Layer API)

Tầng nghiệp vụ chịu trách nhiệm xử lý các thuật toán mật mã đầu cuối, phân tích an toàn thông tin và kết nối RPC mạng lưới:

`AuthService.signInWithGoogle()`

Đầu ra: Promise<GoogleUser>

Nhiệm vụ: Triệu gọi Firebase Auth API để kiểm tra danh tính và chuẩn hóa cấu trúc dữ liệu hồ sơ người dùng Web2.

AuthService.loginWithGoogleService()

Đầu ra: Promise<Object>

Nhiệm vụ: Khởi chạy popup Google, xử lý token để tự động sinh khóa Private Key ảo cố định phục vụ định danh Web3 tự động.

WalletService.connectMetaMask()

Đầu ra: Promise<Object>

Nhiệm vụ: Khởi tạo kết nối mạng RPC lớp thấp nối trực tiếp tới tiện ích mở rộng MetaMask Extension trên trình duyệt.

FaucetService.ensureGas(walletAddress, googleUid, onProgress)

Đầu vào: walletAddress: String, googleUid: String, onProgress: Function

Đầu ra: Promise<Boolean>

Nhiệm vụ: Gửi yêu cầu HTTP POST tới máy chủ Faucet để kiểm tra số dư và tự động tài trợ tiền gas cấp bách cho ví ảo người dùng.

PasswordEvaluator.evaluatePasswordStrength(password, userInputs)

Đầu vào: password: String, userInputs: Array<String>

Đầu ra: PasswordStrengthResult

Nhiệm vụ: Phân tích cấu trúc chuỗi, tính toán độ phức tạp, đối chiếu thông tin cá nhân và trả về kết quả định lượng chi tiết.

PasswordEvaluator.assessVaultPasswords(vaults)

Đầu vào: vaults: Array<VaultItem>

Đầu ra: PasswordAssessmentResult

Nhiệm vụ: Duyệt mảng dữ liệu rõ trên RAM để thống kê số lượng mật khẩu yếu, mật khẩu trùng lặp diện rộng trong hệ thống kết sắt.

CryptoEngine.deriveKey(password, salt, options)

Đầu vào: password: String, salt: String, options: Object

Đầu ra: Promise<CryptoKey>

Nhiệm vụ: Áp dụng thuật toán PBKDF2 với vòng lặp cấu hình cao để dẫn xuất khóa đối xứng AES-256 từ mật khẩu rõ của người dùng.

CryptoEngine.encrypt(data, key, options)

Đầu vào: data: Object, key: CryptoKey, options: Object

Đầu ra: Promise<EncryptedPayload>

Nhiệm vụ: Thực thi thuật toán mã hóa đối xứng mã dịch chuỗi AES-GCM-256 kèm IV

ngẫu nhiên để bảo vệ dữ liệu thô.

`CryptoEngine.decrypt(payload, key)`

Đầu vào: payload: EncryptedPayload, key: CryptoKey

Đầu ra: Promise<Object|String>

Nhiệm vụ: Giải mã chuỗi Ciphertext thu được bằng khóa RAM, trả về cấu trúc mảng hoặc chuỗi tường minh gốc.

`BlockchainService.getVaultCidFromChain(address, provider)`

Đầu vào: address: String, provider: String

Đầu ra: Promise<Object>

Nhiệm vụ: Thực hiện lệnh gọi không tốn phí lên Smart Contract on-chain để đọc liên kết định danh CID IPFS mới nhất.

`BlockchainService.updateVaultCidOnChain(cid, provider, uid)`

Đầu vào: cid: String, provider: String, uid: String

Đầu ra: Promise<Object>

Nhiệm vụ: Đóng gói dữ liệu giao dịch, yêu cầu ký ví để thực hiện ghi đè chuỗi băm CID mới lên không gian lưu trữ của Smart Contract.

Tầng Lưu trữ và Hạ tầng Web3 (Storage Layer API)

Tầng này đảm nhiệm việc thực thi cơ chế lưu trữ lai, đồng bộ hóa trạng thái tệp nén mật mã giữa local và các nút mạng phân tán:

`VaultService.openDatabase(userAddress)`

Đầu vào: userAddress: String

Đầu ra: Promise<IDBDatabase>

Nhiệm vụ: Kiểm tra và khởi tạo cấu trúc kết nối an toàn tới cơ sở dữ liệu IndexedDB NoSQL cục bộ phân tách riêng biệt theo địa chỉ ví.

`VaultService.unlockAndSyncVault(password, addr, provider, uid)`

Đầu vào: Tham số định danh và xác thực đầy đủ

Đầu ra: Promise<Object>

Nhiệm vụ: Phối hợp chuỗi hành động: Đọc Smart Contract lấy CID → Tải file từ mạng IPFS Gateway → Giải mã và nạp dữ liệu lên RAM phiên làm việc.

`VaultService.saveVaultsWithKey(vaults, key, addr, options)`

Đầu vào: Mảng dữ liệu, khóa đối xứng RAM và cấu hình địa chỉ ví

Đầu ra: Promise<Object>

Nhiệm vụ: Đồng bộ song song: Mã hóa dữ liệu → Upload IPFS thu về CID → Ghi CID lên Blockchain → Ghi Ciphertext vào IndexedDB nội bộ thiết bị.

`IpfsService.uploadToIPFS(encryptedData)`

Đầu vào: `encryptedData`: IPFSPayload

Đầu ra: `Promise<String>` (Chuỗi mã hóa hash CID)

Nhiệm vụ: Thực hiện đóng gói và đăng tải tệp tin JSON nén mật mã lên hạ tầng đám mây IPFS thông qua dịch vụ node Pinata.

`IpfsService.fetchFromIPFS(cid)`

Đầu vào: `cid`: String

Đầu ra: `Promise<IPFSPayload>`

Nhiệm vụ: Truy vấn và tải nội dung gói mật mã từ mạng lưới lưu trữ phi tập trung thông qua các cổng Gateway công cộng.

3.5.2 Đặc tả các giao diện lập trình trình duyệt tích hợp (Browser Native APIs)

Bên cạnh các API nội bộ tự phát triển, hệ thống Password Manager ứng dụng trực tiếp các API hệ thống có sẵn của trình duyệt Web (Browser Native APIs) để thực thi các tác vụ tính toán mật mã và lưu trữ ở tầng thấp bảo mật ngay tại phía máy khách (*Client-side*), đảm bảo dữ liệu rõ không bao giờ rời khỏi thiết bị.

Web Crypto API (`window.crypto.subtle`)

Môi trường cung cấp các thuật toán mật mã học cấp độ phần cứng được cách ly an toàn trong tiến trình chạy của trình duyệt:

`crypto.getRandomValues(typedArray)`

Đầu vào: Mảng byte trống cấu trúc định dạng `Uint8Array`.

Đầu ra: Mảng chứa chuỗi các byte ngẫu nhiên bảo mật cao vô điều kiện (Entropy nguồn cao).

Ứng dụng nghiệp vụ: Triệu gọi để sinh muối ngẫu nhiên (*Salt*) và Vector khởi tạo đối xứng (*IV*) cho bộ lõi tiện ích *CryptoEngine*.

`crypto.subtle.digest("SHA-256", data)`

Đầu vào: Dữ liệu văn bản thô đã chuyển đổi cấu trúc sang định dạng mảng byte.

Đầu ra: Mảng byte chứa kết quả băm một chiều bảo mật SHA-256.

Ứng dụng nghiệp vụ: Hỗ trợ băm mật khẩu chủ siêu tốc không đồng bộ thông qua hàm `hashText`.

`crypto.subtle.importKey(...)`

Tham số truyền: Chuỗi mật khẩu dạng rõ thô của người dùng.

Công dụng hệ thống: Nạp an toàn chuỗi mật khẩu thô vào phân vùng bộ nhớ cách ly của

trình duyệt để làm nguyên liệu đầu vào sinh khóa đối xứng.

`crypto.subtle.deriveKey(...)`

Đầu vào: Tham số cấu hình thuật toán PBKDF2 (310,000 vòng lặp, SHA-256, *salt*) và khóa thô đã được nạp qua hàm `importKey`.

Đầu ra: Đối tượng khóa mật mã đối xứng bảo mật `CryptoKey` (AES-256-GCM).

Ứng dụng nghiệp vụ: Dẫn xuất khóa mật mã chính thức từ Master Password đầu vào.

`crypto.subtle.encrypt(...)` | `crypto.subtle.decrypt()`

Đầu vào: Khóa `CryptoKey` đối xứng, Vector khởi tạo IV và mảng byte dữ liệu đích.

Đầu ra: Bản mã Ciphertext đóng gói hoặc bản rõ văn bản thô Plaintext sau khi giải gói thành công.

Ứng dụng nghiệp vụ: Bảo vệ toàn diện dữ liệu mảng cấu trúc kết sắt và thông tin cá nhân trước khi thực hiện lưu trữ ngoại vi.

Hệ quản trị cơ sở dữ liệu IndexedDB API

Nền tảng cơ sở dữ liệu phi quan hệ tích hợp cục bộ hỗ trợ lưu trữ khối dữ liệu Ciphertext dung lượng lớn có cấu trúc:

`indexedDB.open(dbName, version)`

Tham số truyền: Chuỗi định danh tên cơ sở dữ liệu ví dụ `dbName` và phiên bản lược đồ cấu hình.

Công dụng hệ thống: Khởi tạo hoặc thiết lập mở kết nối luồng I/O tới phân vùng lưu trữ IndexedDB cục bộ trên ổ cứng trình duyệt.

`db.transaction([stores], mode)`

Tham số truyền: Mảng danh sách các bảng đích cần tác động (`vaultEncrypted`, `vaultMeta`) và chế độ khóa (`readonly` hoặc `readwrite`).

Công dụng hệ thống: Mở phiên giao dịch an toàn mang tính cô lập, phòng chống xung đột ghi dữ liệu song song tại tầng lưu trữ local.

`store.get(key)` | `store.put(record)` | `store.delete(key)`

Công dụng hệ thống: Thực thi các câu lệnh truy vấn cấu trúc dữ liệu thấp (*Low-level CRUD*) để tiến hành đọc dữ liệu nén, ghi đè Ciphertext hoặc giải phóng xóa bỏ cấu hình kho khóa khỏi thiết bị vật lý.

Giao diện tương tác hệ thống khay nhớ tạm (Clipboard API)

API hỗ trợ tương tác an toàn với các tiến trình bộ nhớ của hệ điều hành máy tính:

`navigator.clipboard.writeText(text)`

Đầu vào: Chuỗi ký tự văn bản rõ của mật khẩu tài khoản sau khi giải mã hoàn chỉnh trên

RAM.

Đầu ra: Trả về một đối tượng Promise<void>.

Ứng dụng nghiệp vụ: Thực thi hành động sao chép mật khẩu an toàn vào khay nhớ tạm (*System Clipboard*) của người dùng ngay khi kích hoạt sự kiện nhấn nút sao chép trên giao diện bảng điều khiển VaultPanel.

3.5.3 Giao diện lập trình ứng dụng HTTP Web API (JIT Gas Faucet Server)

Bên cạnh các API phân tầng nội bộ, hệ thống triển khai một dịch vụ phụ trợ *Faucet Server* (chạy độc lập tại cổng 3001 thông qua tệp tin `faucet.js`).

a) Cấu hình Endpoint và Mô hình Dữ liệu Đầu vào:

- **Phương thức định tuyến:** HTTP POST
- **Đường dẫn Endpoint:** `/api/fund`
- **Công dụng hành động:** Tài trợ tự động một lượng cố định 0.002 Sepolia ETH phí gas vào ví ảo đích để người dùng Web2 có thể tương tác trực tiếp với Smart Contract on-chain mà không cần tự mua hoặc xin token.
- **Định dạng cấu trúc Request Body (JSON):**

Listing 1 Định dạng dữ liệu đầu vào gửi tới Faucet Server

```

1  {
2    "address": "0x4B20993Bc481177ec7E8f571ceCaE8A9e22C02db", // Địa chỉ ví ảo
   ↪   can nap gas
3    "uid": "google-auth-uid-123456789" // UID của tài khoản Google để kiểm
   ↪   soát rate limit
4  }
```

b) Đặc tả các trạng thái phản hồi đầu ra (Response JSON Schemas): Tùy thuộc vào trạng thái số dư hệ thống và hạn mức kiểm soát của định danh người dùng (*rate limit*), máy chủ phụ trợ trả về các mã trạng thái phản hồi được chuẩn hóa cấu trúc như sau:

- **Mã phản hồi 200 OK (Trạng thái nạp gas thành công):** Trả về khi ví ảo có số dư thấp dưới ngưỡng và tài khoản Google chưa vượt quá giới hạn hàng ngày.

Listing 2 Cấu trúc phản hồi 200 OK khi thực hiện nạp gas thành công

```

1 {
2   "success": true,
3   "message": "Da tai tro phi gas (0.002 Sepolia ETH) thanh cong vao vi ao
   ↳ cua ban.",
4   "txHash":
   ↳ "0x5c504e1077501b9166d03b3cc41d5d179646009495b4cf15d11165ef61a29302",
5   "funded": true
6 }

```

- **Mã phản hồi 200 OK (Bỏ qua hành động vì ví đã đủ điều kiện):** Trả về khi hệ thống kiểm tra thấy số dư tài khoản của ví vẫn lớn hơn hoặc bằng 0.001 ETH, không cần phân phối thêm để tiết kiệm tài nguyên cho trạm chung.

Listing 3 Cấu trúc phản hồi 200 OK khi ví đích đã có sẵn số dư an toàn

```

1 {
2   "success": true,
3   "message": "Vi cua ban da co du gas de thuc hien giao dich.",
4   "balance": 0.0015,
5   "funded": false
6 }

```

- **Mã phản hồi 429 Too Many Requests (Vượt quá giới hạn):** Trả về nhằm ngăn chặn hành vi spam hoặc tấn công từ chối dịch vụ (DoS), giới hạn tối đa mỗi định danh Google UID chỉ được phê duyệt nhận gas 3 lần trong vòng 24 giờ.

Listing 4 Cấu trúc phản hồi lỗi 429 do vượt quá tần suất định mức

```

1 {
2   "success": false,
3   "error": "Ban da vuot qua gioi han nhan gas hom nay (toi da 3 lan/ngay).".
4 }

```

- **Mã phản hồi 500 Server Error (Lỗi hệ thống phụ trợ):** Trả về khi kết nối RPC mạng Sepolia bị gián đoạn hoặc tài khoản tổng của trạm cấp phát rơi vào trạng thái cạn kiệt nguồn kinh phí thử nghiệm.

Listing 5 Cấu trúc phản hồi lỗi 500 phát sinh từ phía hệ thống máy chủ

```

1 {
2   "success": false,
3   "error": "Vi tai tro cua tram xang dang het tien Sepolia ETH. Vui long thu
    ↪ lai sau."
4 }

```

3.5.4 Đặc tả các giao diện lập trình thư viện ngoài tích hợp (External Library APIs)

Để tối ưu hóa độ chính xác trong các tác vụ kiểm định an toàn thông tin và đánh giá rủi ro mật mã diện rộng mà không làm tăng độ phức tạp mã nguồn tự phát triển, hệ thống tích hợp các API chuyên dụng được cung cấp từ các thư viện mã nguồn mở uy tín chạy tại môi trường máy khách.

Thư viện phân tích kiểm định mật khẩu (@zxcvbn-ts/core)

`zxcvbnOptions.setOptions(config)`

Đầu vào: Đối tượng cấu hình `config` chứa tệp từ điển ngôn ngữ tự nhiên (*dictionary*) và sơ đồ liên kết vị trí các phím bấm trên bàn phím vật lý (*graphs*).

Đầu ra: Trả về giá trị `void`.

`zxcvbn(password, userInputs)`

Đầu vào: `password: String` (Mật khẩu cần kiểm tra) và `userInputs: Array<String>` (Mảng danh sách các thông tin cá nhân tùy biến của chính người dùng như ngày sinh, số điện thoại, tên đăng nhập).

Đầu ra: Trả về một đối tượng chứa dữ liệu phân tích mật mã học chuyên sâu bao gồm:

- `score: Number`: Thang điểm định lượng từ 0 đến 4 đại diện cho các mức tương ứng từ Rất yếu đến Rất mạnh.
- `feedback: Object`: Khối thông tin chứa chuỗi lý do trực quan tại sao mật khẩu bị đánh giá thấp (*warning*) và mảng các đề xuất hành động cụ thể để nâng cao độ khó mật mã (*suggestions*).
- `crackTimesDisplay: Object`: Ước tính thời gian cần thiết để một máy tính hoặc mạng lưới botnet có thể bẻ khóa thành công mật khẩu ở các kịch bản tốc độ tấn công ngoại tuyến khác nhau.



Để làm rõ phương thức vận hành và cấu trúc thông tin di chuyển qua các tầng kiến trúc trong sơ đồ lớp, hệ thống thực hiện phân tách và đặc tả chi tiết dữ liệu theo ba trạng thái môi trường: trên bộ nhớ RAM (Runtime), lưu trữ cục bộ ngoại tuyến (IndexedDB), và bao gói bảo mật (IPFS/Smart Contract).

Khi kết sắt dữ liệu được người dùng mở khóa thành công, dữ liệu thô dạng văn bản rõ (*plaintext*) sẽ chỉ tồn tại duy nhất trên bộ nhớ RAM nhằm đảm bảo tính an toàn tối đa, tự động giải phóng khi phiên làm việc kết thúc.

a) Cấu trúc bản ghi kết sắt (VaultItem): Mỗi thông tin tài khoản, mật khẩu hoặc ghi chú mật được lưu trữ dưới dạng một đối tượng JavaScript nằm trong mảng `vaults` quản lý bởi `AppContext`:

Bảng 3.13: Đặc tả cấu trúc thực thể VaultItem trên RAM

Thuộc tính	Kiểu dữ liệu	Ý nghĩa / Vai trò	Ví dụ minh họa
<code>id</code>	String	Khóa chính dạng UUID sinh ngẫu nhiên để định danh bản ghi.	"vault-item-uuid-987654"
<code>title</code>	String	Tiêu đề hiển thị của bản ghi mật khẩu trên giao diện.	"Tài khoản GitHub"
<code>category</code>	String	Phân loại mục dữ liệu (logins, cards, notes).	"logins"
<code>username</code>	String	Tên đăng nhập hoặc Email liên kết với tài khoản.	"developer_2026"
<code>password</code>	String	Mật khẩu giải mã dạng Plain-text phục vụ sao chép/hiển thị.	"GitHubPassword#VerySafe99!"
<code>url</code>	String	Địa chỉ trang web liên kết với tài khoản.	"https://github.com"
<code>notes</code>	String	Ghi chú bổ sung (mặc định ẩn trên giao diện).	"Tài khoản làm việc chính"
<code>updatedAt</code>	Number	Mốc thời gian cập nhật bản ghi (Unix timestamp dạng mili giây).	1782245231000

b) Cấu trúc thông tin phiên làm việc (GoogleUser): Đại diện cho danh tính người dùng thuộc phân đoạn Web2, được xác thực thông qua Firebase OAuth và lưu giữ tạm thời:

Bảng 3.14: Đặc tả cấu trúc thông tin người dùng GoogleUser

Thuộc tính	Kiểu dữ liệu	Ý nghĩa / Vai trò	Ví dụ minh họa
uid	String	ID duy nhất cố định của người dùng từ hệ thống Firebase.	"firebase-uid-abcde12345"
email	String	Địa chỉ Gmail đăng ký tài khoản Google.	"nguyenvana@gmail.com"
displayName	String	Tên hiển thị công khai lấy từ Google Profile.	"Nguyễn Văn A"
photoURL	String	Đường dẫn ảnh đại diện (avatar) của tài khoản Google.	"http://googleusercontent.com/..."

c) Khối cấu trúc dữ liệu hỗ trợ nghiệp vụ: Bên cạnh dữ liệu tài khoản, hệ thống định nghĩa các mô hình dữ liệu nội bộ để phục vụ tăng bảo mật xử lý logic:

- **UserProfileData:** Lưu trữ thông tin cá nhân mở rộng (*fullName, dob, phone*) được mã hóa đầu-cuối nhằm ẩn danh tính người dùng trước khi đồng bộ.
- **PasswordStrengthResult:** Định dạng gói tin trả về từ tiện ích *PasswordEvaluator*, chứa các thuộc tính định lượng độ mạnh mật khẩu (*score* từ 0 đến 4, nhãn hiển thị *label*, các cảnh báo an toàn *warning* và danh sách gợi ý *suggestions*).
- **PasswordAssessmentResult:** Cấu trúc dữ liệu tổng hợp phục vụ hiển thị báo cáo sức khỏe bảo mật của toàn bộ kho khóa ứng dụng (đếm tổng số mật khẩu yếu, mật khẩu an toàn và danh sách các mục vi phạm chính sách).

Kiến trúc lưu trữ cục bộ ngoại tuyến (Local IndexedDB Schema)

Tại tầng lưu trữ, ứng dụng triển khai giải pháp cơ sở dữ liệu phi quan hệ (*NoSQL*) IndexedDB cục bộ trên trình duyệt. Nhằm đảm bảo tính cô lập dữ liệu giữa các tài khoản Web3 khác nhau, cơ sở dữ liệu được khởi tạo động độc lập theo cấu trúc tên miền ví: `vault-db-${userAddress.toLowerCase}`

a) Store bản ghi mã hóa (vaultEncrypted): Mục lưu trữ này tuân thủ nguyên tắc tối giản, chỉ chứa duy nhất một bản ghi mang khóa chính id: "primary" đại diện cho toàn bộ kho khóa đã được mã hóa cấu trúc thành một chuỗi nén an toàn:

Listing 6 Cấu trúc bao gói dữ liệu nén trong store vaultEncrypted

```

1  {
2    "id": "primary",
3    "payload": {
4      "version": 1,
5      "algorithm": "AES-256-GCM",
6      "payloadType": "json",
7      "iv": "c3VwZXJfaXZfZ2VuZXJhdGVkMTI=",
8      "cipherText": "U2FsdGVkX194c2Q4Zjk4...[chuoi base64 do dai lon]..."
9    },
10   "updatedAt": 1782245231000
11  }

```

b) Store siêu dữ liệu (vaultMeta): Đảm nhiệm vai trò lưu trữ các tham số cấu hình phi tập trung và các cờ trạng thái đồng bộ hóa phục vụ dịch vụ *VaultService*:

Bảng 3.15: Đặc tả các khóa cấu hình trong store vaultMeta

Khóa (key)	Kiểu dữ liệu	Ý nghĩa nội dung và vai trò hệ thống
kdfSaltV1	String (Base64)	Muối ngẫu nhiên (16 bytes) dùng dẫn xuất khóa PBKDF2 bảo mật cục bộ.
migratedFromLocalStorageV1	Boolean	Cờ đánh dấu trạng thái chuyển đổi thành công từ LocalStorage cũ.
lastSyncedCid	String	Mã băm CID IPFS mới nhất đã được đồng bộ khớp với Blockchain.
pendingSyncV1	Object null	Vùng đệm lưu giữ gói tin lỗi và dữ liệu chưa đồng bộ khi mạng lưới nghẽn.

Định dạng gói dữ liệu mã hóa trao đổi (IPFS & Export Envelopes)

Khi truyền dữ liệu ra khỏi biên giới thiết bị cục bộ sang mạng lưới phân tán IPFS (thông qua *IpfsService*) hoặc đóng gói thành file tải về (thông qua *ImportExportPage*), dữ liệu buộc phải bọc trong các bao mật mã tiêu chuẩn (*Cryptographic Envelopes*).

a) Gói dữ liệu lưu trữ IPFS (IPFSPayload): Tập tin JSON đẩy lên IPFS chứa đầy đủ thông tin mật mã tự giải gói (*Self-contained decryption package*) mà không lưu trữ bất kỳ thành phần nhạy cảm nào ở dạng rõ:

Listing 7 Cấu trúc tệp tin mật mã IPFSPayload

```

1  {
2      "encryptedPayload": {
3          "version": 1,
4          "algorithm": "AES-256-GCM",
5          "payloadType": "json",
6          "iv": "c3VwZXJfaXZfZ2VuZXJhdGVkMTI=",
7          "cipherText": "U2FsdGVkX194c2Q4Zjk4..."
8      },
9      "salt": "dGVzdF9zYWx0XzEyMzQ1Ng=="
10 }

```

Trong đó, cipherText mang mảng dữ liệu mật mã liên kết Array<VaultItem> và thông tin cá nhân UserProfileData. Thuộc tính salt đóng vai trò cung cấp hạt giống toán học độc lập để các thiết bị khác khi tải về có thể tái tạo chính xác khóa đối xứng khi người dùng nhập đúng mật khẩu chủ.

b) Gói tệp tin sao lưu thủ công (JSON Export File): Khi người dùng kết xuất file sao lưu cá nhân, cấu trúc gói tin được định nghĩa chặt chẽ bao gồm định danh định dạng format và cấu hình thuật toán hàm dẫn xuất khóa kdf tách biệt:

Listing 8 Định dạng cấu trúc tệp tin sao lưu kết xuất ngoại vi

```

1  {
2      "format": "vault-ciphertext-v1",
3      "kdf": {
4          "algorithm": "PBKDF2-SHA256",
5          "iterations": 310000,
6          "salt": "ZXhwb3J0X3NhbHRfZ2VuZXJhdGVkMTI="
7      },
8      "cipher": {
9          "version": 1,
10         "algorithm": "AES-256-GCM",
11         "payloadType": "json",
12         "iv": "ZXhwb3J0X2l2XzEyYn10ZXM=",
13         "cipherText": "U2FsdGVkX19leHBvcnRfZGF0YQ=="
14     }
15 }

```

Cấu trúc trạng thái trên Hợp đồng thông minh (Solidity Storage Layer)

Để tối ưu hóa chi phí tài nguyên lưu trữ mạng lưới phi tập trung, hợp đồng thông minh VaultPointer.sol trên Blockchain Sepolia được thiết kế nhằm duy trì dữ liệu ở mức tối giản hóa tối đa, chỉ lưu trữ liên kết định danh.

a) Struct dữ liệu con trỏ kết sắt (VaultData): Hợp đồng định nghĩa cấu trúc dữ liệu tinh gọn để đóng gói thông tin định vị:

Listing 9 Định nghĩa cấu trúc Struct dữ liệu trên Solidity

```

1 struct VaultData {
2     string cid;           // Chuỗi IPFS CID (đo dài từ 46 đến 59 ký tự)
3     uint256 updatedAt; // Mốc thời gian ghi nhận khối block.timestamp (tính
    ↪   sang giây)
4 }
```

b) Khai báo không gian trạng thái Hợp đồng: Sử dụng cấu trúc ánh xạ bảng băm liên kết (*mapping*) được bảo vệ quyền truy cập chặt chẽ để quản lý tập người dùng ứng dụng:

Listing 10 Cấu trúc lưu trữ trạng thái mapping trên Smart Contract

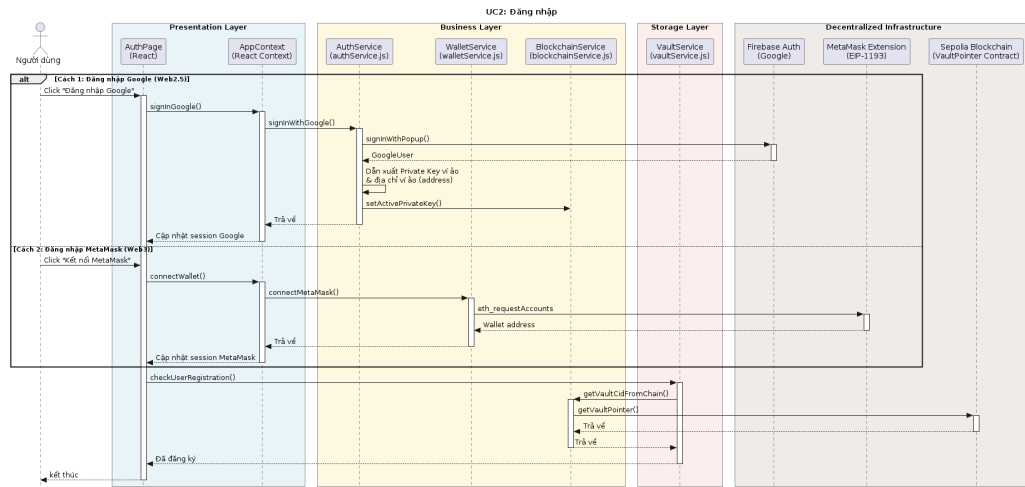
```

1 mapping(address => VaultData) private userVaults;
```

3.8 Thiết kế hệ thống biểu đồ tuần tự (Sequence Diagrams)

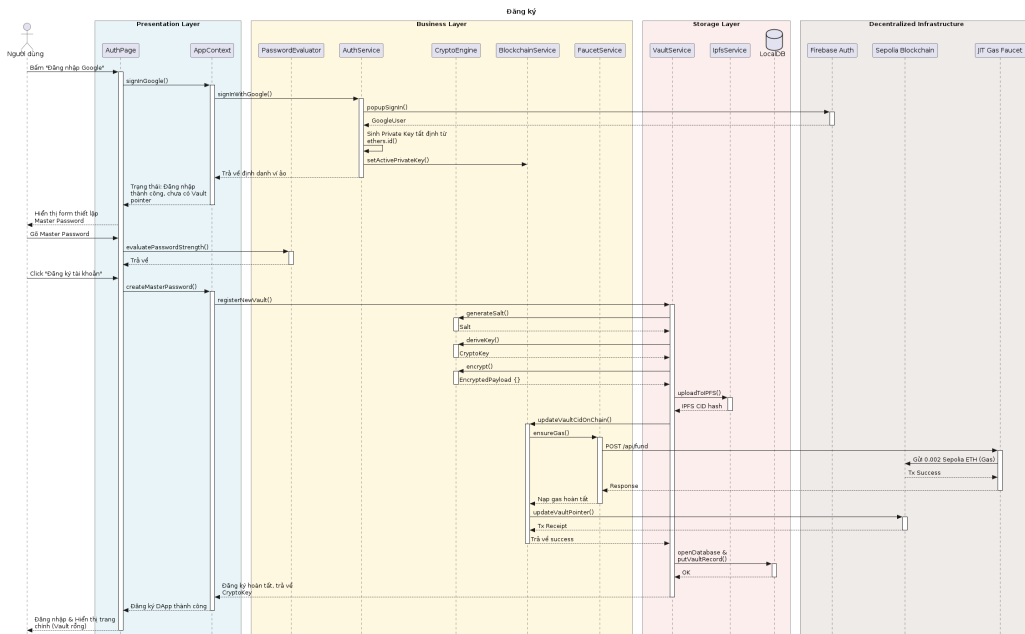
Tài liệu báo cáo chỉ tập trung xây dựng biểu đồ tuần tự cho các luồng nghiệp vụ đặc trưng

Đăng nhập



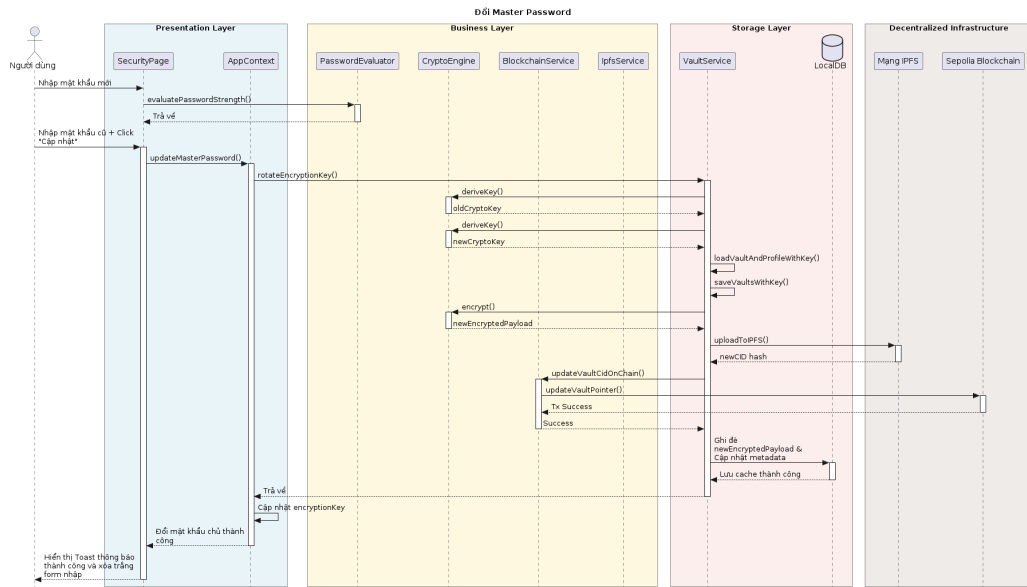
Hình 3.7: Biểu đồ tuần tự luồng hành động Đăng nhập hệ thống

Đăng ký



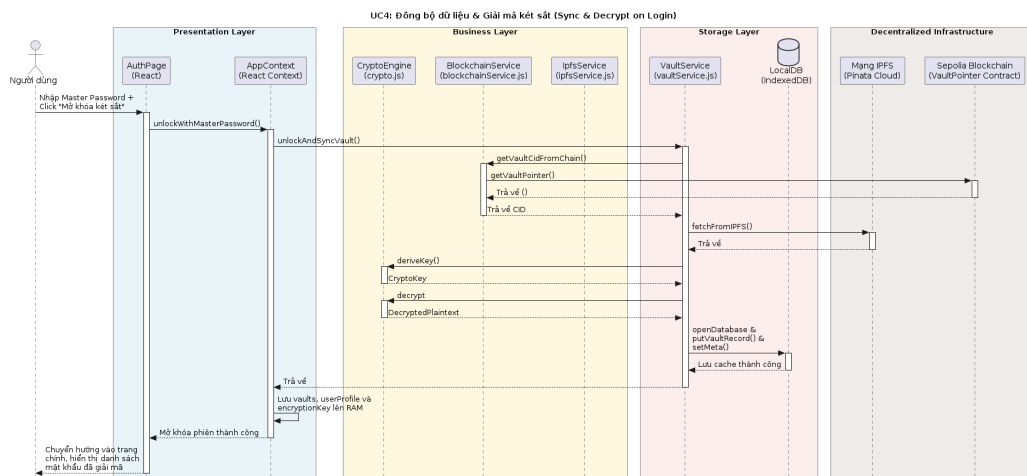
Hình 3.8: Biểu đồ tuần tự luồng hành động Đăng ký tài khoản mới

Đổi mật khẩu



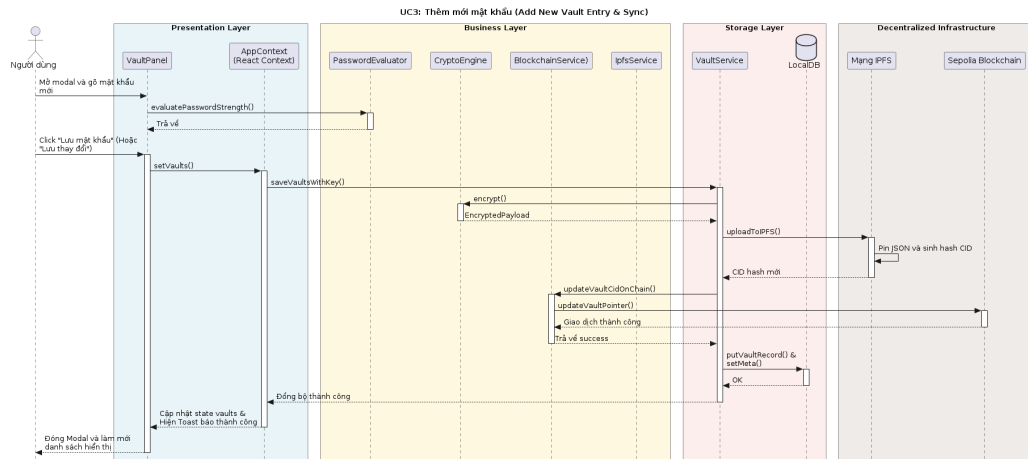
Hình 3.9: Biểu đồ tuần tự luồng hành động Thay đổi mật khẩu chủ

Đồng bộ và giải mã



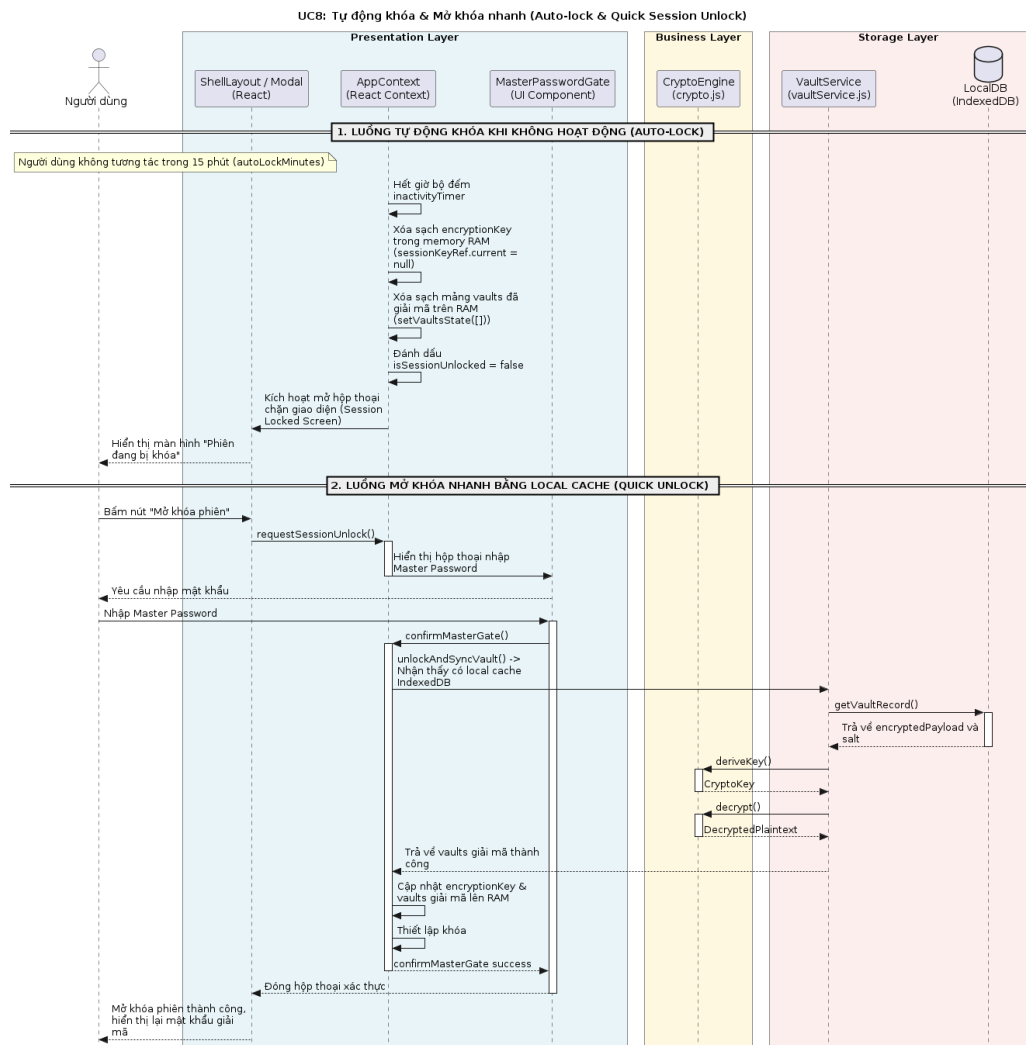
Hình 3.10: Biểu đồ tuần tự luồng hành động Đồng bộ và Giải mã kết nối

Thêm mật khẩu



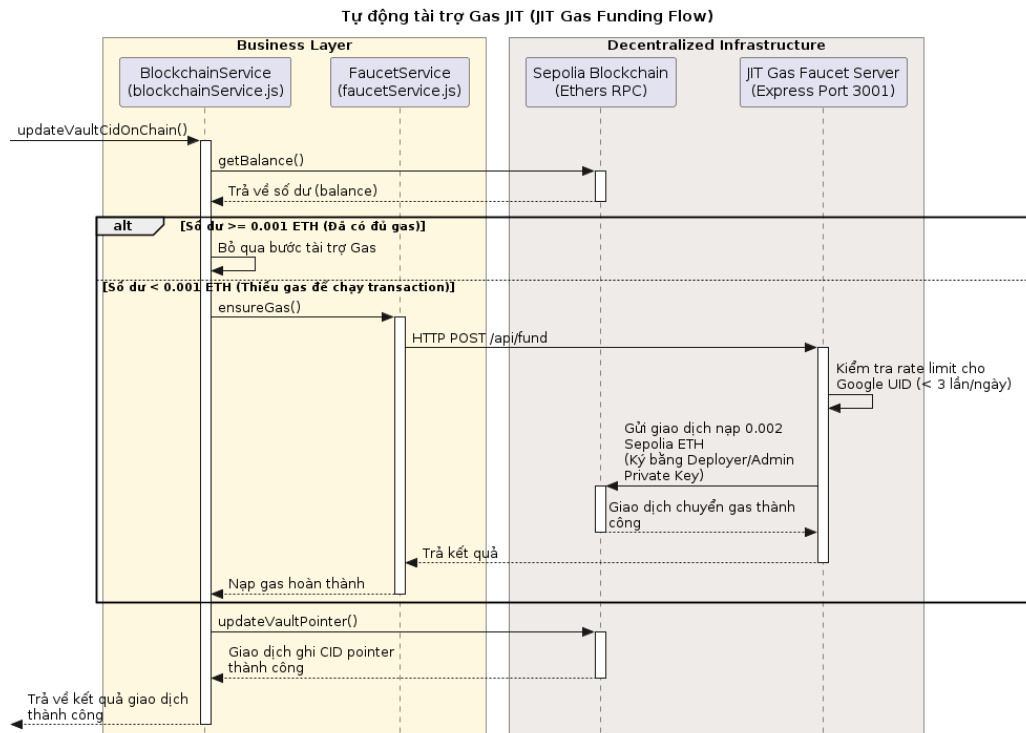
Hình 3.11: Biểu đồ tuần tự luồng hành động Thêm bản ghi mật khẩu mới

Khóa phiên



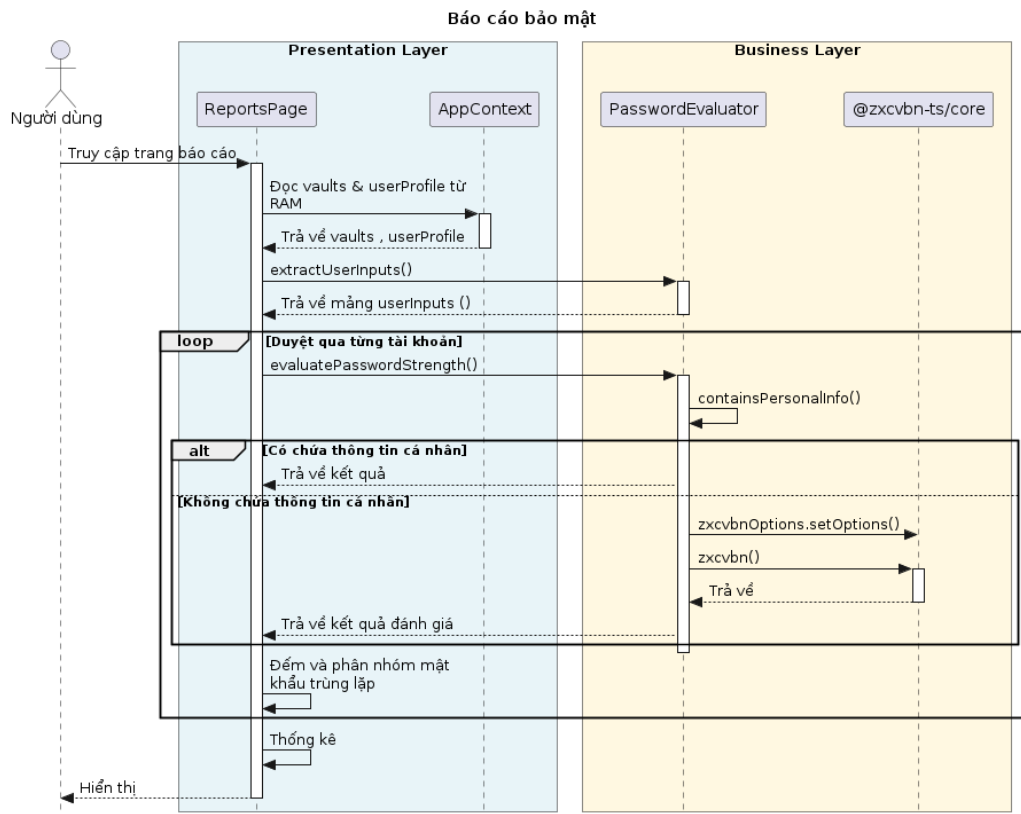
Hình 3.12: Biểu đồ tuần tự luồng hành động Khóa phiên làm việc tự động

Tài trợ gas tự động



Hình 3.13: Biểu đồ tuần tự luồng hành động Tài trợ phí Gas tự động

Báo cáo



Hình 3.14: Biểu đồ tuần tự luồng hành động Kết xuất báo cáo bảo mật

Chương 4

Kết quả cài đặt và thử nghiệm

Chương này trình bày chi tiết kết quả cài đặt, triển khai và thử nghiệm hệ thống ứng dụng quản lý mật khẩu DApp Password Manager.

4.1 Môi trường cài đặt và triển khai

4.1.1 Môi trường phần cứng

Hệ thống được cài đặt và thử nghiệm trên cấu hình phần cứng như bảng 4.1.

Bảng 4.1: Cấu hình phần cứng môi trường phát triển

Thành phần	Cấu hình
Hệ điều hành	Windows 11 Pro 64-bit
Bộ xử lý (CPU)	Intel Core i5
Bộ nhớ RAM	16 GB DDR5
Ổ cứng	SSD 512 GB
Kết nối mạng	Internet băng thông rộng (Sepolia Testnet và Pinata IPFS Gateway)

4.1.2 Môi trường phần mềm

Bảng 4.2 liệt kê các phần mềm và công nghệ được sử dụng trong quá trình phát triển và triển khai hệ thống.

Bảng 4.2: Môi trường phần mềm và công cụ phát triển

Phần mềm / Công cụ	Mô tả	Phiên bản
Node.js	Môi trường thực thi JavaScript phía server	LTS (v20.x)
npm	Trình quản lý gói thư viện JavaScript	v10.x
Visual Studio Code	Trình soạn thảo mã nguồn	Mới nhất
Google Chrome / Edge	Trình duyệt hỗ trợ Web Crypto API	Mới nhất
MetaMask Extension	Ví điện tử Ethereum trên trình duyệt	Mới nhất
Git	Hệ thống quản lý phiên bản mã nguồn	v2.x
Docker Desktop	Nền tảng đóng gói container	Mới nhất

4.1.3 Công nghệ và thư viện chính

Bảng 4.3 trình bày chi tiết ngăn xếp công nghệ (tech stack) được sử dụng trong dự án.

Bảng 4.3: Ngăn xếp công nghệ chính của hệ thống

Lớp	Công nghệ	Vai trò
Frontend	React 19, Vite 8	Xây dựng giao diện SPA hiệu năng cao
Styling	Tailwind CSS 3	Thiết kế giao diện responsive nhanh chóng
Routing	React Router v7	Quản lý điều hướng trang ứng dụng SPA
Mã hóa	Web Crypto API	Mã hóa AES-256-GCM, dẫn xuất khóa PBKDF2
Đánh giá mật khẩu	@zxcvbn-ts/core v3	Phân tích entropy và chấm điểm độ mạnh mật khẩu cục bộ
Blockchain	Ethers.js v6	Tương tác Smart Contract trên mạng Ethereum
Smart Contract	Solidity v0.8.19	Viết hợp đồng thông minh VaultPointer
Biên dịch & Deploy	Hardhat v2.28	Biên dịch, kiểm thử và triển khai Smart Contract
Xác thực	Firebase Auth v12	Đăng nhập bằng tài khoản Google
Lưu trữ phân tán	IPFS (Pinata API)	Lưu trữ dữ liệu mã hóa phi tập trung
Bộ đệm cục bộ	IndexedDB	Cache dữ liệu mã hóa offline tại trình duyệt
Đóng gói	Docker Compose	Triển khai multi-container (Frontend + Faucet)

4.2 Kết quả cài đặt hệ thống

4.2.1 Cài đặt và khởi chạy ứng dụng

Quá trình cài đặt hệ thống được thực hiện qua các bước sau:

1. **Cài đặt thư viện phụ thuộc:** Chạy lệnh `npm install` tại thư mục gốc dự án để tải toàn bộ các gói thư viện được khai báo trong `package.json`, bao gồm 7 thư viện runtime (React, Ethers, Firebase, zxcvbn,...) và 7 thư viện phát triển (Hardhat, Vite, Tailwind CSS,...).
2. **Cấu hình biến môi trường:** Tạo file `.env` từ mẫu `.env.example`, điền đầy đủ các thông số: Firebase SDK config (7 biến), Pinata JWT token, khóa riêng tư ví triển khai (Deployer Private Key), và địa chỉ RPC Sepolia.
3. **Biên dịch Smart Contract:** Chạy lệnh `npm run compile` để biên dịch mã nguồn Solidity. Kết quả: “Compiled 1 Solidity file successfully”.
4. **Triển khai Smart Contract lên Sepolia Testnet:** Chạy lệnh `npm run deploy:sepolia`. Hệ thống tự động ký giao dịch, deploy hợp đồng, in ra địa chỉ contract và ghi tự động vào file `.env`.
5. **Khởi chạy JIT Gas Station Faucet:** Chạy lệnh `npm run faucet` để khởi động máy chủ cấp phí gas tự động tại cổng 3001.
6. **Khởi chạy Frontend:** Chạy lệnh `npm run dev` để khởi động Vite dev server tại `http://localhost:`



```

PS C:\Users\PC\Desktop\Đạt\Password_manager_dapp_develop> npm run compile

> exercise2_wp@2.0.0 compile
> hardhat compile

✓ Help us improve Hardhat with anonymous crash reports & basic usage data? (Y/n) - y
Downloading compiler 0.8.19
Compiled 1 Solidity file successfully (evm target: paris).
PS C:\Users\PC\Desktop\Đạt\Password_manager_dapp_develop>

```

Hình 4.1: Kết quả biên dịch Solidity triển khai Smart Contract VaultPointer lên mạng Sepolia Testnet

```

PS C:\Users\PC\Desktop\Đạt\Password_manager_dapp_develop> npm run deploy:sepolia

> exercise2_wp@2.0.0 deploy:sepolia
> hardhat run scripts/deploy.js --network sepolia

Deploying VaultPointer contract to Sepolia...
✓ VaultPointer deployed successfully!
Contract Address: 0xe96E1A80259f3131365B1FacA99101B53895a1b4
Sepolia Explorer: https://sepolia.etherscan.io/address/0xe96E1A80259f3131365B1FacA99101B53895a1b4

Updating .env file...
Updated .env with VITE_VM_PRIVATE_KEY=...
✓ .env updated with correct values

Exporting ABI...
Exported ABI to src\contracts\VaultPointer.abi.json
✓ ABI exported to src/contracts/VaultPointer.abi.json

Deployment complete!
Assertion failed: !(handle->flags & UV_HANDLE_CLOSING), file src\win\async.c, line 76
PS C:\Users\PC\Desktop\Đạt\Password_manager_dapp_develop>

```

Hình 4.2: Kết quả triển khai Smart Contract VaultPointer lên mạng Sepolia Testnet

```

PS D:\JJin\Lap_trinh\bootstrap project\exercise2_wp> npm run dev
> exercise2_wp@2.0.0 dev
> vite

10:35:38 PM [vite] (client) hmr update /src/layouts/ShellLayout.jsx, /src/styles/index.css
10:38 PM [vite] (client) hmr update /src/layouts/ShellLayout.jsx, /src/styles/index.css

PS D:\JJin\Lap_trinh\bootstrap project\exercise2_wp> npm run faucet
> exercise2_wp@2.0.0 faucet
> node server/faucet.js

JIT Gas Station Faucet listening on port 3001

```

Hình 4.3: Khởi chạy đồng thời JIT Gas Station Faucet (cổng 3001) và Frontend Vite (cổng 5173)

4.2.2 Triển khai Smart Contract trên Blockchain

Smart Contract VaultPointer đã được triển khai thành công trên mạng Sepolia Testnet của Ethereum. Bảng 4.4 tóm tắt thông tin triển khai.

Bảng 4.4: Thông tin triển khai Smart Contract VaultPointer

Thông tin	Giá trị
Tên contract	VaultPointer
Ngôn ngữ	Solidity v0.8.19
Mạng blockchain	Sepolia Testnet (Chain ID: 11155111)
Địa chỉ contract	0x... (được tự động ghi vào .env sau deploy)
Hàm ghi (write)	updateVaultPointer(string _cid)
Hàm đọc (view)	getVaultPointer(address _user)
Gas trung bình (ghi)	~0.00001 SepoliaETH - 0.000117 SepoliaETH
Gas trung bình (đọc)	~không tốn phí

 0x8e38b1caf50... 	0x60806040	11067619	8 days ago	0xF93C903E...cF6f526bE 	 OUT	 Contract Creation 	0 ETH	0.00076102
 0x19940db5a6... 	0x60806040	11067555	8 days ago	0xF93C903E...cF6f526bE 	 OUT	 Contract Creation 	0 ETH	0.0006197
 0x1bdc1e5882... 	Transfer	11067403	8 days ago	0x993a0f36...63293adD9 	 IN	0xF93C903E...cF6f526bE 	0.05 ETH	0.00002221

Hình 4.4: Triển khai thành công

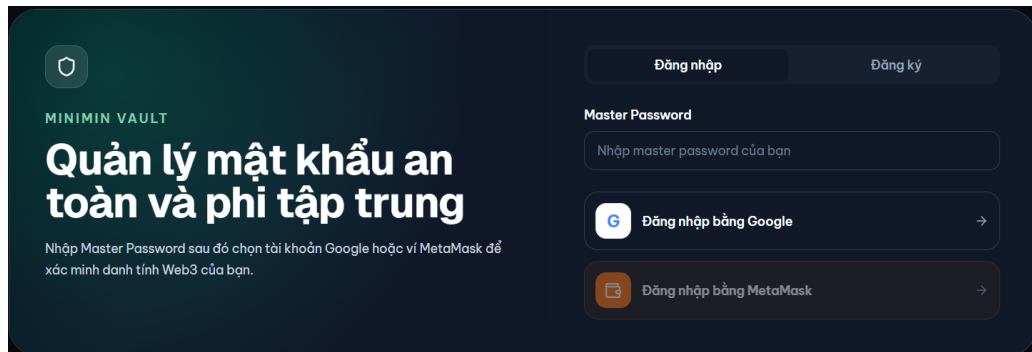
4.3 Kết quả giao diện người dùng

Giao diện người dùng của ứng dụng DApp Password Manager được xây dựng theo phong cách thiết kế hiện đại, hỗ trợ chế độ sáng/tối (dark mode) và đáp ứng đầy đủ trên nhiều kích thước màn hình. Dưới đây là kết quả các trang chính.

4.3.1 Trang xác thực đăng nhập (AuthPage)

Trang xác thực cung cấp hai phương thức đăng nhập:

- **Đăng nhập bằng Google:** Sử dụng Firebase Authentication, tự động sinh ví Web3 tất định (deterministic wallet) từ UID và email.
- **Kết nối ví MetaMask:** Kết nối trực tiếp với ví MetaMask đã cài đặt trên trình duyệt.



Hình 4.5: Giao diện trang xác thực đăng nhập của ứng dụng

4.3.2 Trang thiết lập Master Password

Sau khi đăng nhập lần đầu, người dùng được yêu cầu thiết lập Master Password. Hệ thống tích hợp bộ đánh giá độ mạnh mật khẩu @zxcvbn-ts hiển thị trực quan thanh đo cường độ mật khẩu, thời gian dự đoán bị bẻ khóa, và các gợi ý cải thiện.

Đăng nhập Đăng ký

Tạo Master Password

...

PASSWORD STRENGTH Cần cải thiện

Yếu

Chính sách: Mật khẩu master phải đạt mức Khá trở lên và tối thiểu 8 ký tự.

Cảnh báo: sequences

Gợi ý: anotherWord

Ước tính bẻ khóa: 1tgiây

Thông tin tối ưu hóa bảo mật mật khẩu (Tùy chọn) Mở rộng

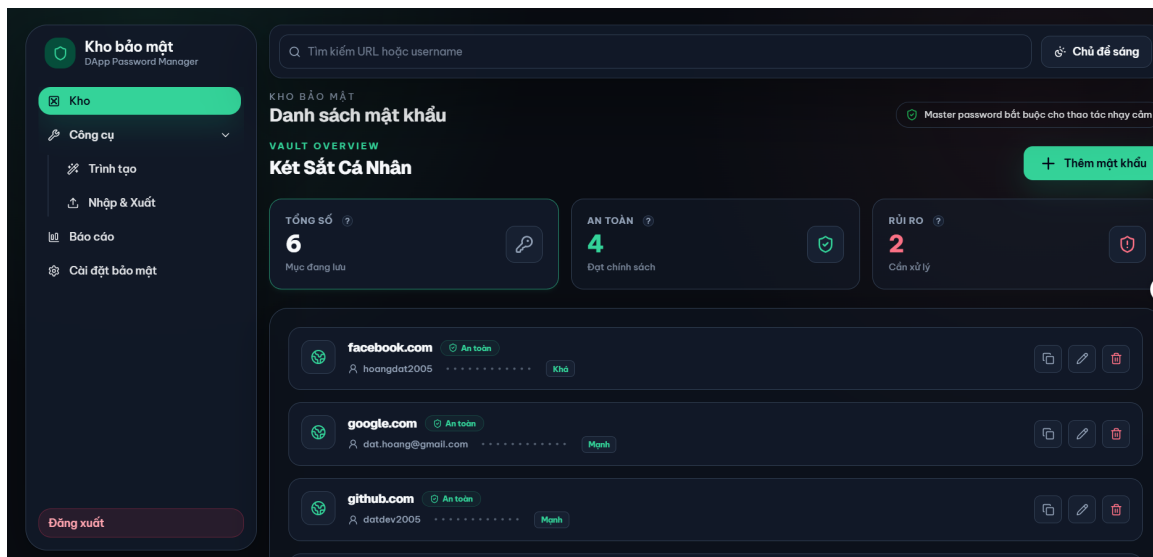
Xác nhận Master Password

...

Hình 4.6: Giao diện thiết lập Master Password với đánh giá cường độ zxcvbn

4.3.3 Trang kết sắt quản lý mật khẩu (VaultPage)

Đây là trang chính của ứng dụng, cho phép người dùng thực hiện đầy đủ các thao tác CRUD (Create, Read, Update, Delete) trên danh sách mật khẩu. Giao diện hiển thị danh sách các tài khoản đã lưu với khả năng tìm kiếm, lọc theo danh mục, sao chép mật khẩu nhanh và ẩn/hiện mật khẩu.



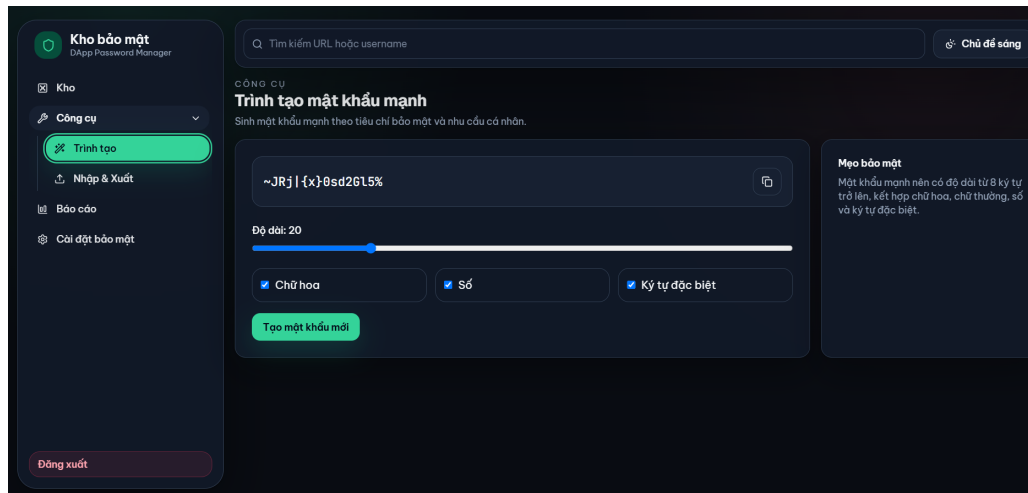
Hình 4.7: Giao diện trang kho quản lý mật khẩu chính

The screenshot shows the 'Thêm mật khẩu mới' (Add New Password) form. It has a title bar with a close button. The form contains three main sections: 'WEBSITE URL' with a text input field containing 'https://example.com'; 'USERNAME HOẶC EMAIL' with a text input field containing 'name@example.com'; and 'MẬT KHẨU' (Password) with a text input field containing 'Nhập mật khẩu' and a toggle icon. Below these is a dashed box containing the text 'Đánh giá độ mạnh và thời gian bẻ khóa ước tính.' (Estimated strength and cracking time). At the bottom is a large green button labeled 'Lưu thay đổi' (Save Changes).

Hình 4.8: Giao diện thêm mật khẩu mới vào kết sắt

4.3.4 Trang sinh mật khẩu ngẫu nhiên (GeneratorPage)

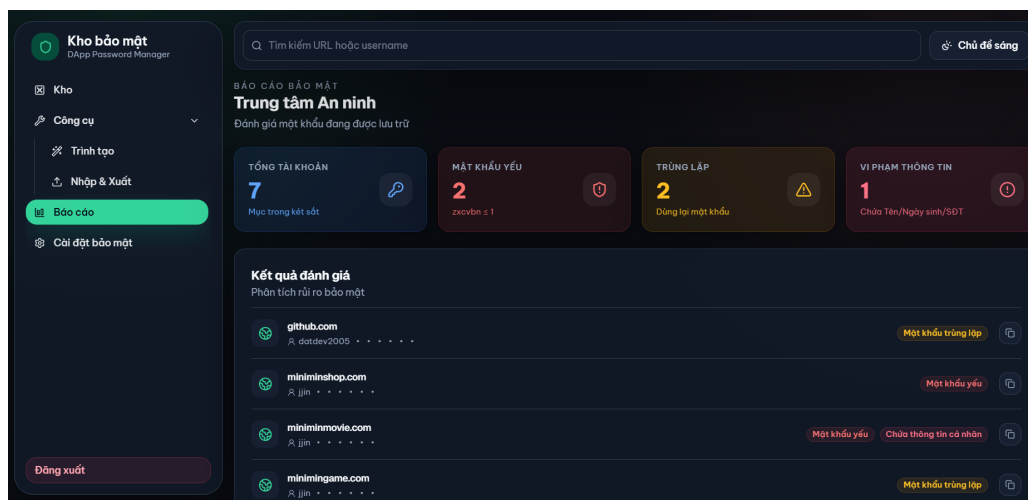
Hệ thống cung cấp công cụ sinh mật khẩu ngẫu nhiên an toàn, cho phép người dùng tùy chỉnh độ dài và các tập ký tự (chữ thường, chữ hoa, số, ký tự đặc biệt).



Hình 4.9: Giao diện sinh mật khẩu ngẫu nhiên với tùy chỉnh độ dài và tập ký tự

4.3.5 Trang báo cáo an toàn (ReportsPage)

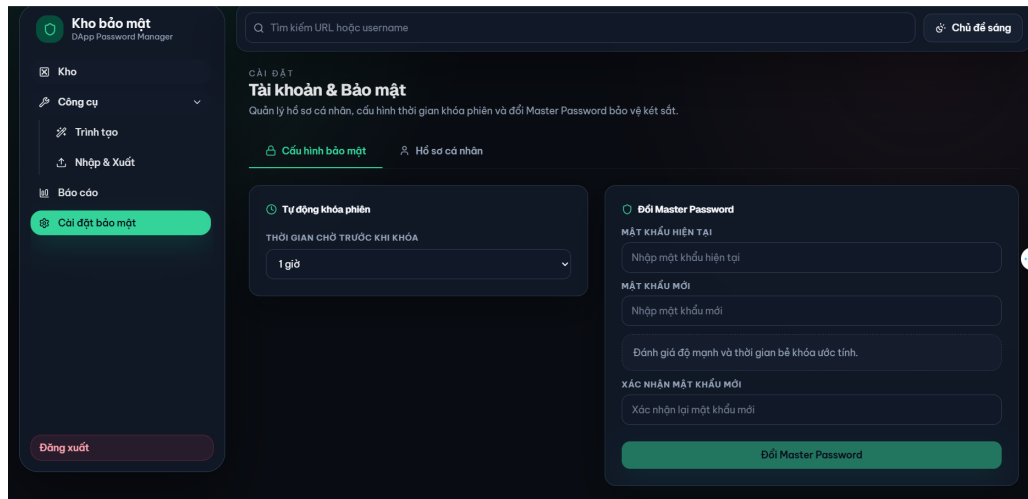
Trang báo cáo phân tích toàn bộ mật khẩu trong kết sắt, đưa ra cảnh báo về các mật khẩu yếu, mật khẩu trùng lặp và các tài khoản có nguy cơ bảo mật.



Hình 4.10: Giao diện trang báo cáo an toàn phân tích mật khẩu

4.3.6 Trang cài đặt bảo mật (SecurityPage)

Trang cài đặt cho phép người dùng thay đổi Master Password (Key Rotation), cấu hình thời gian tự động khóa kết sắt (Auto-Lock), và quản lý nhập/xuất dữ liệu.

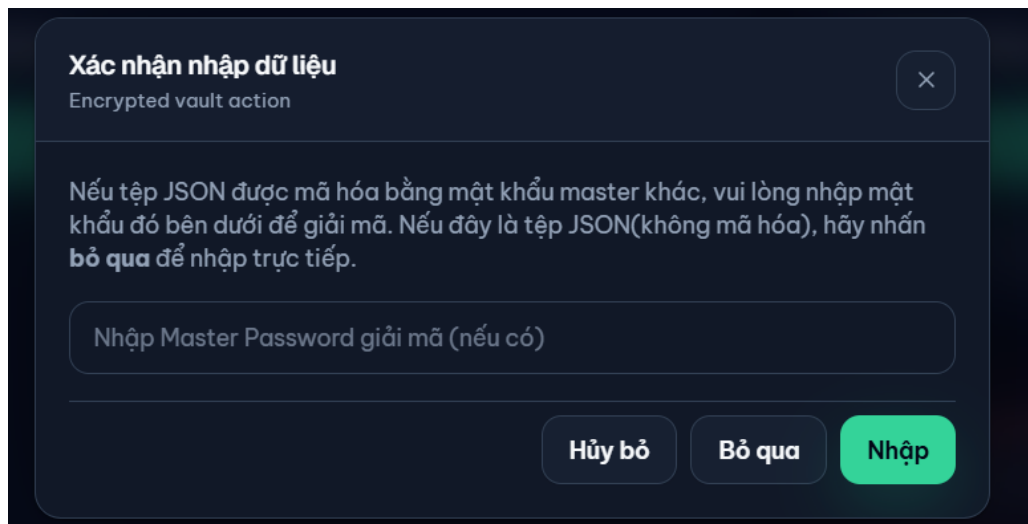


Hình 4.11: Giao diện trang cài đặt bảo mật

4.3.7 Trang nhập/xuất dữ liệu (ImportExportPage)

Hệ thống hỗ trợ nhập và xuất dữ liệu kết sắt dưới hai định dạng:

- **Định dạng vault-ciphertext-v1:** File JSON đã mã hóa AES-256-GCM, yêu cầu Master Password để giải mã khi import.
- **Định dạng plaintext legacy:** File JSON mẩu dạng thô (không mã hóa), hỗ trợ tương thích ngược.



Hình 4.12: Giao diện nhập/xuất dữ liệu kết sắt

4.4 Kết quả thử nghiệm chức năng

Quá trình thử nghiệm được chia thành 10 giai đoạn, bao phủ toàn bộ các tầng chức năng từ mã hóa cục bộ đến đồng bộ blockchain xuyên thiết bị.

4.4.1 Thử nghiệm mã hóa cục bộ và IndexedDB

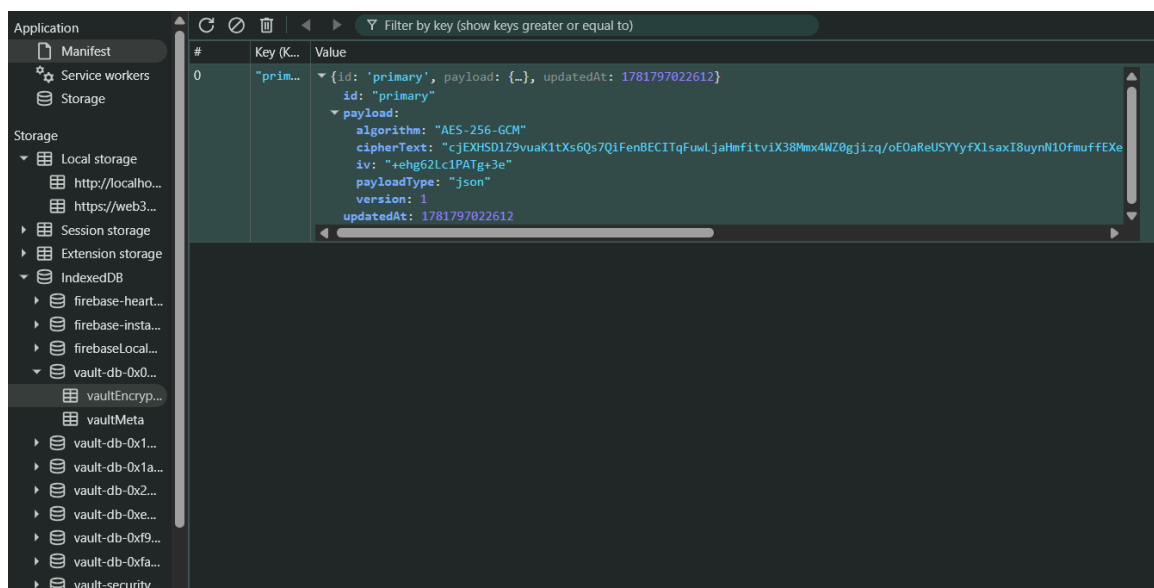
Mục tiêu: Xác minh rằng dữ liệu kết sắt được mã hóa đúng cách bằng AES-256-GCM và lưu trữ an toàn vào IndexedDB của trình duyệt.

Quy trình thử nghiệm:

1. Đăng nhập ứng dụng, tạo Master Password với độ mạnh ≥ 3 (theo thang điểm zxcvbn).
2. Thêm một bản ghi mật khẩu mẫu (URL: <https://example.com>, Username: testuser@example.com).
3. Mở Chrome DevTools → Tab “Application” → “IndexedDB” để kiểm tra dữ liệu.

Kết quả:

- Object Store vaultEncrypted: Bản ghi chứa payload là đối tượng JSON mã hóa với các trường iv (base64), cipherText (base64), algorithm: "AES-256-GCM", version: 1.
- Object Store vaultMeta: Bản ghi kdfSaltV1 chứa salt ngẫu nhiên 16 bytes dùng cho PBKDF2.
- Local Storage: Khóa masterPasswordHash lưu trữ verifier dạng đối tượng JSON với algorithm: "PBKDF2-SHA256", iterations: 310000, salt và hash – **không lưu plaintext Master Password**.



Đánh giá: *Đạt yêu cầu.* Dữ liệu kết sất hoàn toàn được mã hóa trước khi lưu trữ. Không có bất kỳ thông tin mật khẩu dạng plaintext nào xuất hiện trong IndexedDB hay Local Storage.

4.4.2 Thử nghiệm giao diện và thao tác CRUD

Mục tiêu: Kiểm tra hoạt động đầy đủ các thao tác Create, Read, Update, Delete trên giao diện người dùng.

Kết quả thử nghiệm CRUD:

Bảng 4.5: Kết quả thử nghiệm thao tác CRUD trên giao diện

Thao tác	Mô tả kiểm thử	Kết quả
Create	Thêm 5 bản ghi mật khẩu với URL, username, password khác nhau	Đạt
Read	Tìm kiếm bản ghi theo URL hoặc username, lọc theo danh mục	Đạt
Update	Sửa đổi password của bản ghi, lưu lại và kiểm tra cập nhật	Đạt
Delete	Xóa bản ghi, xác nhận modal và kiểm tra danh sách cập nhật	Đạt
Copy	Nhấn nút sao chép mật khẩu, kiểm tra clipboard	Đạt
Toggle visibility	Ẩn/hiện mật khẩu trong bảng danh sách	Đạt



Hình 4.13: Kết quả thử nghiệm các thao tác CRUD trên giao diện kết sất

4.4.3 Thử nghiệm đánh giá độ mạnh mật khẩu

Hệ thống sử dụng thư viện @zxcvbn-ts/core để đánh giá cường độ mật khẩu cục bộ (không gửi dữ liệu lên mạng). Bảng 4.6 trình bày kết quả thử nghiệm.

Bảng 4.6: Kết quả thử nghiệm đánh giá cường độ mật khẩu bằng zxcvbn

Mật khẩu thử	Điểm	Phản hồi giao diện	Đánh giá
123456	0/4	Rất yếu – Mật khẩu rất phổ biến	Đạt
password1	1/4	Yếu – Dễ đoán	Đạt
Abc@2024	2/4	Trung bình	Đạt
Tr0p!c@1Thund3r#2026	4/4	Rất mạnh – Ước tính bẻ khóa: hàng thế kỷ	Đạt

Chỉnh sửa tài khoản

Encrypted vault action

×

WEBSITE URL

miniminshop.com

USERNAME HOẶC EMAIL

jjin

MẬT KHẨU

...

👁

PASSWORD STRENGTH

Cần cải thiện

Yếu

Chính sách: Mật khẩu lưu trong két sắt nên đạt mức Khá trở lên và tối thiểu 8 ký tự.

Cảnh báo: sequences

Gợi ý: anotherWord

Ước tính bẻ khóa: 1tgiây

Lưu thay đổi

Hình 4.14: Kết quả đánh giá cường độ mật khẩu: yếu

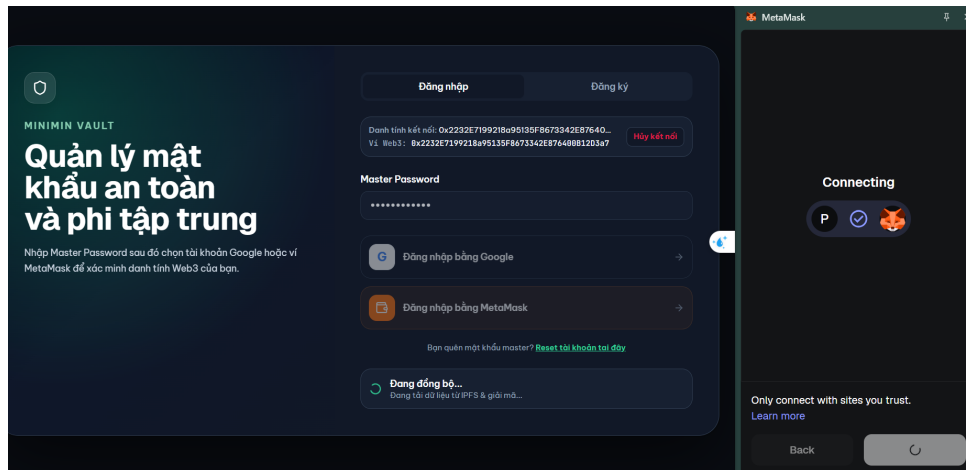
Hình 4.15: Kết quả đánh giá cường độ mật khẩu: mạnh)

4.4.4 Thử nghiệm kết nối MetaMask

Mục tiêu: Xác minh kết nối ví MetaMask hoạt động đúng, lấy được địa chỉ ví và xác nhận mạng Sepolia.

Kết quả:

- Khi nhấn “Kết nối ví MetaMask”, popup MetaMask hiển thị yêu cầu xác nhận kết nối.
- Sau khi xác nhận, ứng dụng hiển thị đúng địa chỉ ví bắt đầu bằng 0x. . . .
- Chain ID xác nhận là 0xaa36a7 (Sepolia = 11155111 thập phân).
- Khi chuyển sang mạng khác (Ethereum Mainnet), hệ thống cảnh báo “Wrong network”.



Hình 4.16: Kết nối ví MetaMask với ứng dụng DApp Password Manager

4.4.5 Thử nghiệm đăng nhập Google và sinh ví ảo tự động

Mục tiêu: Xác minh luồng đăng nhập Google qua Firebase Auth và quá trình sinh ví Web3 tất định.

Kết quả:

- Popup Google Auth hiển thị đúng, người dùng chọn tài khoản Google để đăng nhập.
- Hệ thống tự động sinh cặp khóa Web3 tất định (deterministic wallet) từ:
`ethers.id("dapp_secret_salt_" + uid + email).`
- Ví ảo được tạo có địa chỉ Ethereum hợp lệ (bắt đầu bằng 0x).
- JIT Gas Station Faucet tự động kiểm tra số dư ví ảo và tài trợ 0.002 Sepolia ETH khi số dư dưới 0.001 ETH.

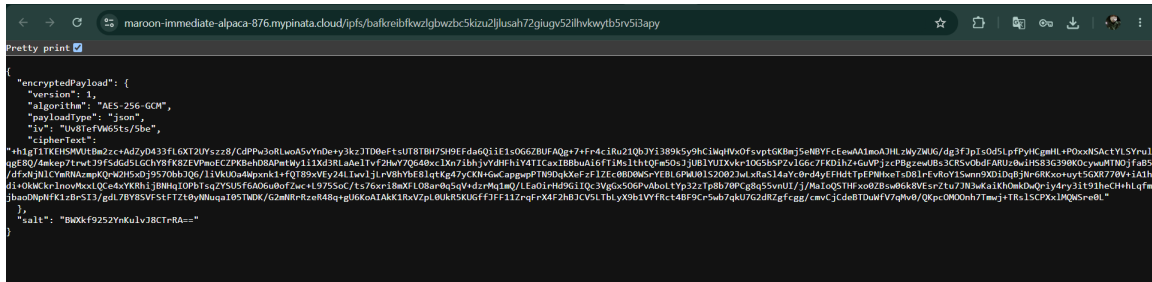
4.4.6 Thử nghiệm tải lên và tải xuống dữ liệu IPFS

Mục tiêu: Xác minh quy trình upload dữ liệu mã hóa lên IPFS thông qua Pinata API và tải xuống qua IPFS Gateway.

Kết quả:

- Khi lưu mật khẩu mới, hệ thống mã hóa toàn bộ kết sất bằng AES-256-GCM, sau đó đẩy payload mã hóa lên IPFS qua Pinata API.
- Response trả về chứa CID (Content Identifier) hợp lệ, ví dụ: `bafkrei...`
- Truy cập gateway `https://gateway.pinata.cloud/ipfs/{CID}` xác nhận nội dung trả về là JSON mã hóa chứa các trường `encryptedPayload` và `salt`.

- Không có bất kỳ dữ liệu plaintext nào xuất hiện trong payload tải lên IPFS.



Hình 4.17: Kết quả upload dữ liệu mã hóa lên IPFS qua Pinata API

4.4.7 Thử nghiệm ghi và đọc CID trên Blockchain

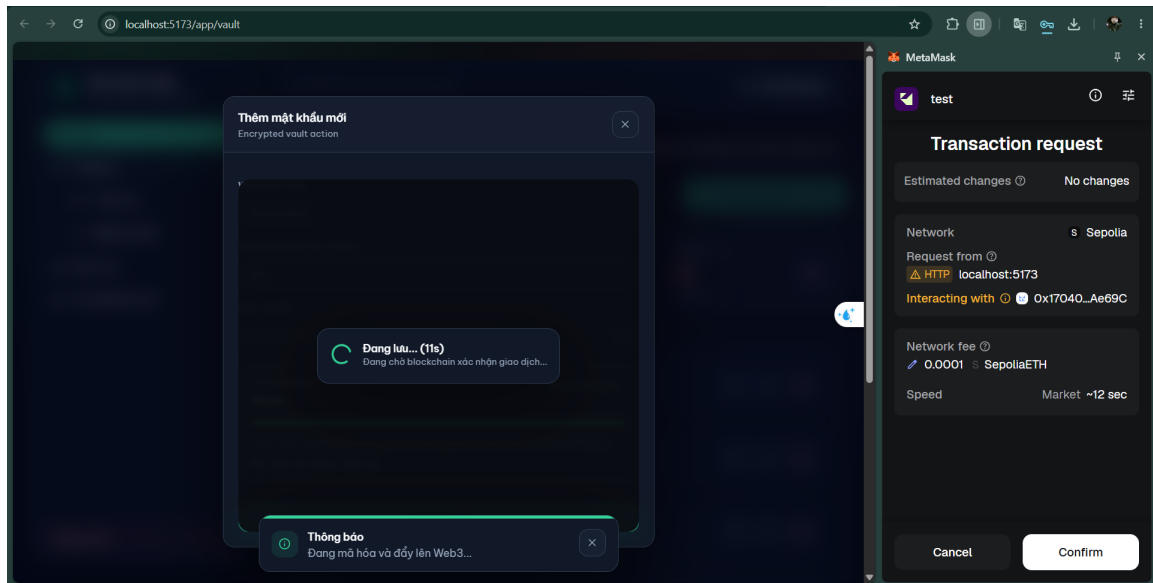
Mục tiêu: Xác minh CID được ghi thành công lên Smart Contract VaultPointer trên Sepolia và có thể đọc lại chính xác.

Quy trình:

1. Sau khi upload IPFS thành công, hệ thống gọi hàm `updateVaultPointer(cid)` trên Smart Contract.
2. MetaMask hiển thị popup yêu cầu ký giao dịch (hoặc ký tự động nếu dùng ví ảo Google).
3. Giao dịch được xác nhận trên blockchain, trả về `txHash` và `blockNumber`.
4. Hàm đọc `getVaultPointer(address)` trả về CID khớp chính xác với CID vừa ghi.

Kết quả:

- Giao dịch ghi CID thành công, trạng thái “Success” trên Sepolia Etherscan.
- Event `VaultUpdated` được phát ra với đúng thông tin user, CID và timestamp.
- Hàm đọc trả về CID chính xác, xác nhận tính toàn vẹn dữ liệu trên blockchain.



Hình 4.18: Giao dịch ghi CID pointer lên Smart Contract VaultPointer thành công trên Sepolia

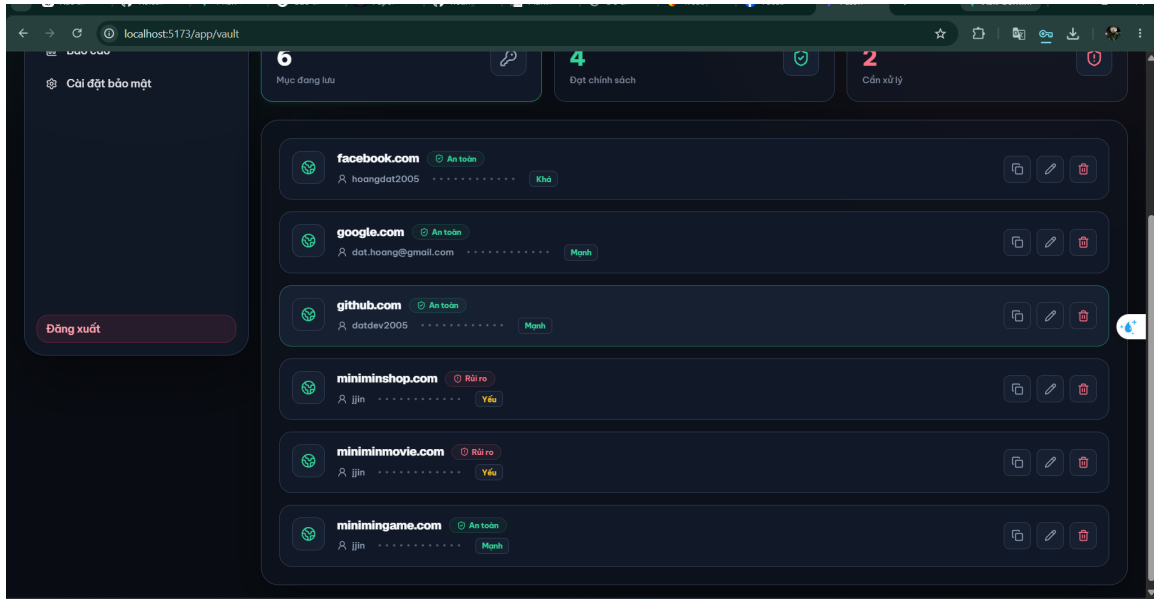
4.4.8 Thử nghiệm đồng bộ xuyên thiết bị (Cross-Device Sync)

Mục tiêu: Mô phỏng kịch bản sử dụng thực tế: người dùng lưu mật khẩu trên thiết bị A, sau đó truy cập và khôi phục dữ liệu trên thiết bị B.

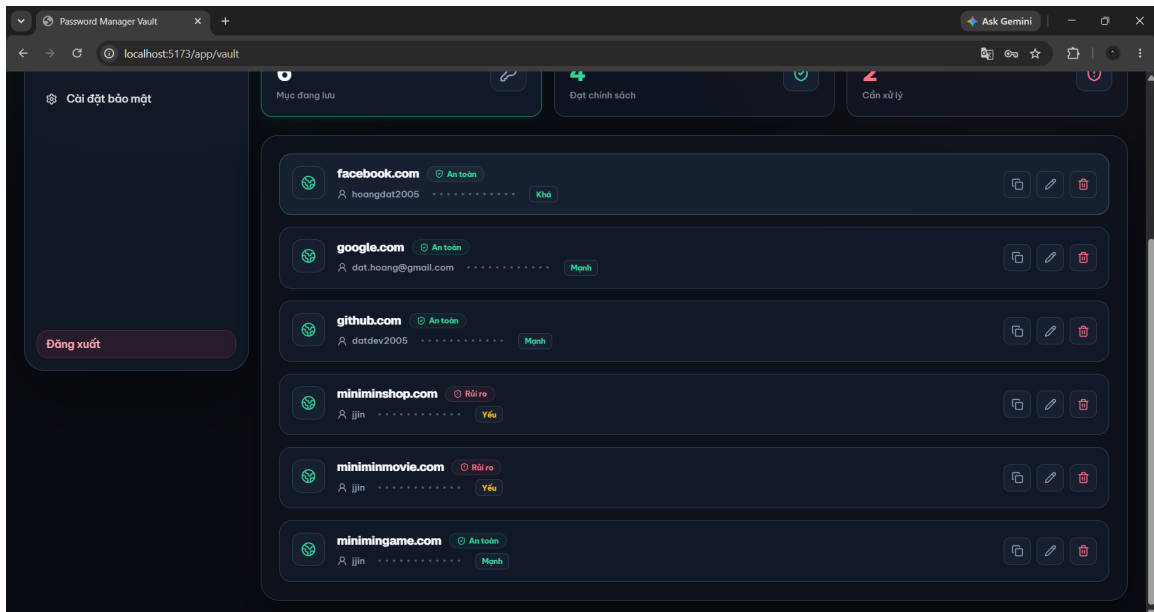
Quy trình mô phỏng:

1. **Thiết bị A:** Đăng nhập, thêm 3 bản ghi mật khẩu mẫu, nhấn “Lưu”. Hệ thống mã hóa → upload IPFS → ghi CID lên blockchain.
2. **Thiết bị B (mô phỏng):** Mở tab ẩn danh (Incognito) trên cùng trình duyệt. IndexedDB hoàn toàn trống.
3. **Thiết bị B:** Đăng nhập cùng tài khoản → Nhập Master Password → Hệ thống đọc CID từ blockchain → Tải payload mã hóa từ IPFS → Giải mã bằng Master Password → Hiện thị đầy đủ 3 bản ghi mật khẩu.

Kết quả: Đạt yêu cầu. Dữ liệu được đồng bộ hoàn toàn và chính xác giữa hai phiên trình duyệt khác nhau thông qua blockchain và IPFS mà không cần máy chủ trung gian.



Hình 4.19: Kết quả đồng bộ xuyên thiết bị 1



Hình 4.20: Kết quả đồng bộ xuyên thiết bị 2

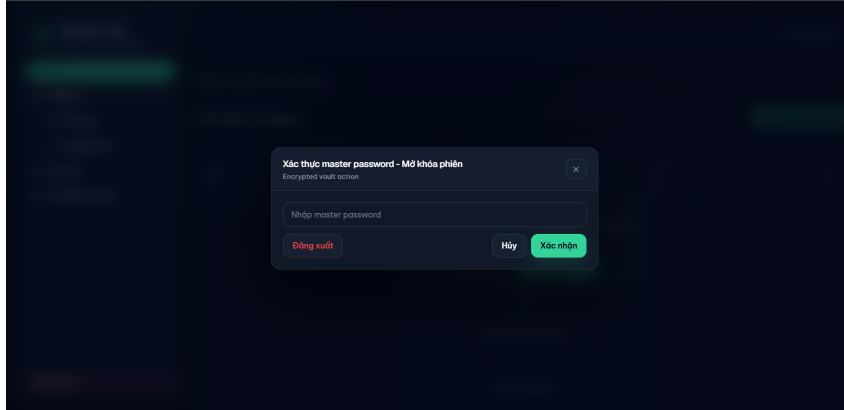
4.4.9 Thử nghiệm tính năng tự động khóa (Auto-Lock)

Mục tiêu: Xác minh khóa mã hóa trên RAM bị xóa sạch đúng thời điểm.

Kết quả:

- Sau khoảng thời gian không tương tác (mặc định 15 phút), hệ thống tự động khóa kết sắt, hiển thị modal “Mở khóa phiên”.

- Khi đóng tab hoặc nhấn F5, sessionKey trong biến tham chiếu RAM bị xóa (gán null), yêu cầu nhập lại Master Password.
- Nút “Khóa kết sắt” trên thanh điều hướng hoạt động chính xác.



Hình 4.21: Modal mở khóa phiên sau khi tính năng Auto-Lock kích hoạt

4.4.10 Thử nghiệm đổi Master Password (Key Rotation)

Mục tiêu: Xác minh quy trình xoay khóa mã hóa hoạt động chính xác.

Quy trình:

1. Vào **Cài đặt** → **Bảo mật** → Nhập Master Password hiện tại và Master Password mới.
2. Hệ thống giải mã toàn bộ kết sắt bằng khóa cũ trên RAM, sinh khóa dẫn xuất mới từ mật khẩu mới, mã hóa lại kết sắt và đẩy bản mã mới lên IPFS & Blockchain.

Kết quả: Đạt yêu cầu. Sau khi đổi Master Password:

- Mật khẩu cũ không thể mở khóa kết sắt (hệ thống báo lỗi “Master Password không chính xác”).
- Mật khẩu mới mở khóa thành công, dữ liệu kết sắt nguyên vẹn.
- CID mới được ghi lên blockchain, phản ánh bản mã hóa mới.

4.4.11 Thử nghiệm nhập/xuất dữ liệu (Import/Export)

Kịch bản 1: Import file plaintext legacy

- Chọn file data_example/mau_plaintext_legacy.json chứa mảng JSON dạng thô.
- Nhấn “Bỏ qua (Skip)” tại modal xác nhận.

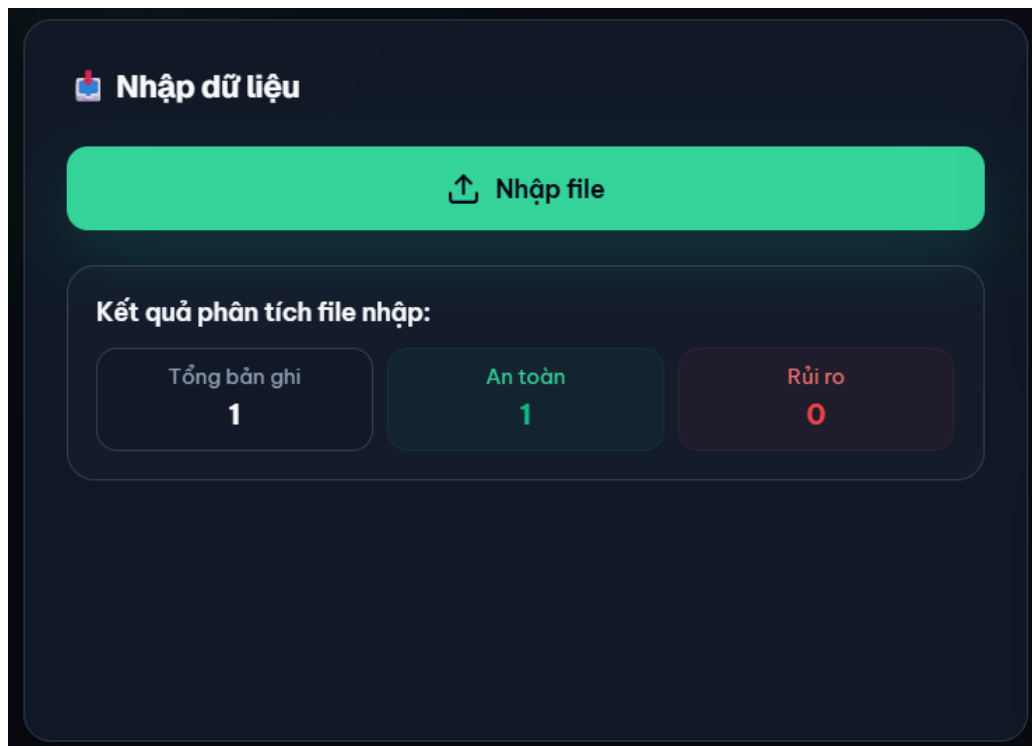
- Kết quả: Toàn bộ mật khẩu mẫu (Facebook, Google, GitHub) được import thành công, mã hóa tự động và đồng bộ Web3.

Kịch bản 2: Import file ciphertext v1

- Chọn file `data_example/mau_ciphertext_v1.json` (file đã mã hóa AES-256-GCM + PBKDF2).
- Nhập mật khẩu giải mã → Hệ thống giải mã, gộp vào kết sắt hiện tại và đồng bộ.
- Kết quả: Import thành công, dữ liệu hiển thị đầy đủ trong kết sắt.

Kịch bản 3: Export kết sắt

- Nhấn “Xuất dữ liệu”, nhập Master Password để xác nhận.
- Hệ thống sinh file JSON định dạng `vault-ciphertext-v1` chứa dữ liệu mã hóa.
- Kết quả: File JSON tải xuống thành công, nội dung hoàn toàn là ciphertext.



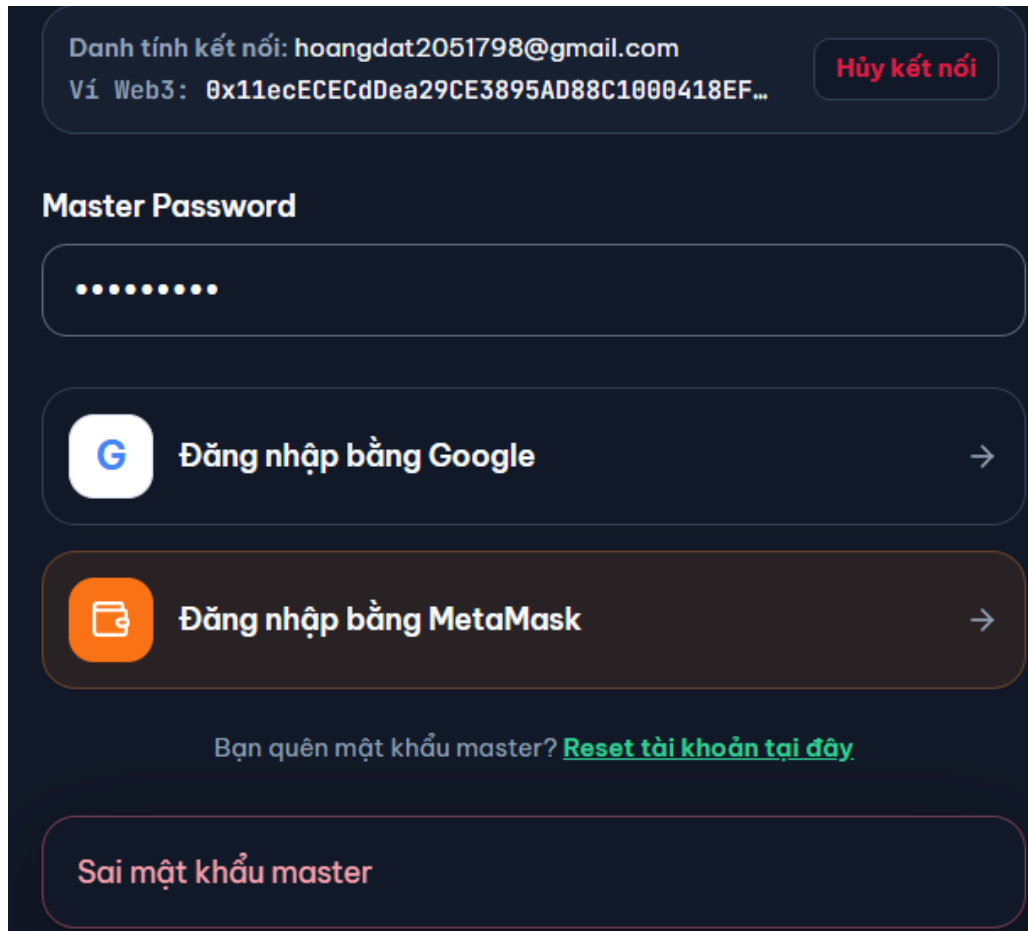
Hình 4.22: Kết quả import dữ liệu mẫu vào kết sắt

4.5 Thử nghiệm xử lý lỗi và tình huống ngoại lệ

Bảng 4.7 trình bày kết quả thử nghiệm các kịch bản lỗi và ngoại lệ.

Bảng 4.7: Kết quả thử nghiệm xử lý lỗi và tình huống ngoại lệ

STT	Kịch bản lỗi	Hành vi hệ thống	Kết quả
1	Nhập sai Master Password	Hiển thị thông báo “Master Password không chính xác”, không giải mã dữ liệu	Đạt
2	Không cấu hình Pinata JWT	Vault lưu local (IndexedDB), bỏ qua IPFS upload, hiển thị cảnh báo	Đạt
3	Từ chối giao dịch MetaMask	Giao dịch không gửi, hiển thị lỗi “User rejected”, dữ liệu lưu local fallback	Đạt
4	Sai mạng blockchain (Network mismatch)	Hiển thị cảnh báo “Wrong network”, yêu cầu chuyển về Sepolia	Đạt
5	IPFS Gateway không khả dụng	Retry tự động 3 lần với exponential backoff, nếu thất bại thì lưu local fallback	Đạt
6	RPC node Sepolia không ổn định	Retry tự động 3 lần, hiển thị trạng thái retry cho người dùng	Đạt
7	Faucet hết tiền Sepolia ETH	Thông báo “Ví tài trợ đang hết tiền”, yêu cầu thử lại sau	Đạt
8	Vượt giới hạn Faucet (>3 lần/ngày)	Thông báo “Đã vượt giới hạn nhận gas hôm nay” (HTTP 429)	Đạt



Hình 4.23: Ví dụ xử lý lỗi và phản hồi trên giao diện người dùng

4.6 Thử nghiệm hiệu năng

4.6.1 Hiệu năng mã hóa và giải mã

Bảng 4.8 trình bày kết quả đo thời gian mã hóa/giải mã với các khối lượng dữ liệu khác nhau.

Bảng 4.8: Kết quả đo hiệu năng mã hóa/giải mã AES-256-GCM với PBKDF2 (310,000 vòng)

Số bản ghi	Thời gian mã hóa	Thời gian giải mã	Ghi chú
10 bản ghi	~0.8 giây	~0.8 giây	Dẫn xuất khóa PBKDF2 chiếm phần lớn thời gian
50 bản ghi	~0.9 giây	~0.9 giây	AES-GCM thực hiện rất nhanh trên Web Crypto API
100 bản ghi	~1.0 giây	~1.0 giây	Hiệu năng ổn định, chấp nhận được
200 bản ghi	~1.2 giây	~1.2 giây	Thời gian chủ yếu đến từ PBKDF2 310K iterations

Nhận xét: Thời gian mã hóa và giải mã chủ yếu đến từ quá trình dẫn xuất khóa PBKDF2 với 310,000 vòng lặp (đáp ứng tiêu chuẩn OWASP). Thao tác AES-256-GCM trên Web Crypto API (thực thi trực tiếp trong nhân trình duyệt) cực kỳ nhanh, thời gian tăng không đáng kể khi tăng số lượng bản ghi.

4.6.2 Hiệu năng đồng bộ Web3

Bảng 4.9: Thời gian trung bình cho các bước đồng bộ Web3

Bước	Thời gian trung bình	Ghi chú
Upload IPFS (Pinata)	~1–3 giây	Phụ thuộc kích thước payload và băng thông mạng
Ghi CID lên blockchain	~15–30 giây	Phụ thuộc tốc độ xác nhận block trên Sepolia
Đọc CID từ blockchain	< 1 giây	Hàm view, không tốn phí gas
Tải dữ liệu từ IPFS	~1–2 giây	Qua IPFS Gateway của Pinata
Tổng luồng đồng bộ	~20–40 giây	Từ lúc nhấn “Lưu” đến xác nhận blockchain

4.7 Thử nghiệm bảo mật

4.7.1 Kiểm tra an toàn dữ liệu trên đường truyền

Bảng 4.10: Kết quả kiểm tra bảo mật dữ liệu trên đường truyền mạng

Kiểm tra	Mô tả	Kết quả
Master Password trên Network	Kiểm tra tất cả request HTTP/HTTPS trong Dev-Tools Network tab	Không tìm thấy
Plaintext mật khẩu trên IPFS	Kiểm tra payload POST tới Pinata API	Chỉ chứa ciphertext
Dữ liệu nhạy cảm trên blockchain	Kiểm tra event logs trên Etherscan	Chỉ chứa CID pointer
Khóa mã hóa trong localStorage	Kiểm tra toàn bộ Local Storage	Không lưu khóa mã hóa

4.7.2 Phân tích mô hình đe dọa (Threat Model)

Bảng 4.11 trình bày kết quả phân tích các kịch bản tấn công và biện pháp phòng vệ tương ứng.

Bảng 4.11: Phân tích mô hình đe dọa và biện pháp phòng vệ

Mối đe dọa	Biện pháp phòng vệ	Đánh giá
IPFS bị xâm nhập, kẻ tấn công lấy được file mã hóa	Dữ liệu hoàn toàn là ciphertext AES-256-GCM. Không thể giải mã nếu không biết Master Password	An toàn
Blockchain công khai, bất kỳ ai đọc được CID	CID chỉ là con trỏ tới file mã hóa, không chứa dữ liệu nhạy cảm	An toàn
Truy cập IndexedDB nội bộ trình duyệt	IndexedDB chỉ lưu bản ciphertext, không chứa plaintext mật khẩu	An toàn
Tấn công brute-force Master Password	PBKDF2 với 310,000 vòng lặp SHA-256 + Salt ngẫu nhiên 16 bytes chống GPU/ASIC	An toàn
Tấn công spam JIT Gas Station Faucet	Rate limit 3 lần/ngày/Google UID, chỉ cấp gas khi số dư < 0.001 ETH	An toàn
Đánh cắp session key trong RAM	Session key tự động xóa khi đóng tab, reload trang, logout hoặc auto-lock	An toàn

4.8 Tổng hợp kết quả thử nghiệm

Bảng 4.12 tổng hợp kết quả thử nghiệm toàn bộ các chức năng chính của hệ thống.

Bảng 4.12: Tổng hợp kết quả thử nghiệm các chức năng chính

STT	Chức năng / Tiêu chí thử nghiệm	Kết quả
1	Mã hóa AES-256-GCM + PBKDF2 dữ liệu kết sắt cục bộ	Đạt
2	Lưu trữ Master Password dạng verifier (không plaintext)	Đạt
3	Giao diện CRUD quản lý mật khẩu	Đạt
4	Đánh giá cường độ mật khẩu bằng zxcvbn	Đạt
5	Sinh mật khẩu ngẫu nhiên an toàn	Đạt
6	Kết nối ví MetaMask trên mạng Sepolia	Đạt
7	Đăng nhập Google và sinh ví ảo tự động	Đạt
8	JIT Gas Station Faucet tài trợ phí gas	Đạt
9	Biên dịch và triển khai Smart Contract VaultPointer	Đạt
10	Upload/Download dữ liệu mã hóa qua IPFS (Pinata)	Đạt
11	Ghi/Đọc CID pointer trên blockchain	Đạt
12	Đồng bộ xuyên thiết bị (Cross-Device Sync)	Đạt
13	Auto-Lock và quản lý session key trên RAM	Đạt
14	Đổi Master Password (Key Rotation)	Đạt
15	Nhập/Xuất dữ liệu (Import/Export)	Đạt
16	Xử lý lỗi và tình huống ngoại lệ	Đạt
17	Bảo mật dữ liệu trên đường truyền	Đạt
18	Hiệu năng mã hóa/giải mã với 200 bản ghi	Đạt
19	Triển khai Docker Container	Đạt
20	Tương thích ngược dữ liệu cũ (legacy format)	Đạt

4.9 Đánh giá và nhận xét

4.9.1 Ưu điểm

- Bảo mật theo nguyên tắc Zero-Knowledge:** Master Password không bao giờ rời khỏi trình duyệt của người dùng. Toàn bộ quá trình mã hóa/giải mã diễn ra trên client-side bằng Web Crypto API, đảm bảo rằng ngay cả nhà phát triển hệ thống cũng không thể truy cập dữ liệu plaintext.
- Kiến trúc Web2.5 lai ghép:** Sự kết hợp giữa tiện ích Web2 (đăng nhập Google, giao diện trực quan) và tính phi tập trung Web3 (IPFS, blockchain) mang lại trải nghiệm người dùng thuận tiện mà vẫn đảm bảo quyền sở hữu dữ liệu.
- Tiêu chuẩn mã hóa cao:** Sử dụng PBKDF2 với 310,000 vòng lặp (đáp ứng khuyến nghị OWASP 2023) kết hợp AES-256-GCM cung cấp mức bảo mật mạnh mẽ chống lại tấn

công brute-force bằng phần cứng GPU/ASIC.

4. **Khả năng chịu lỗi (Fault Tolerance):** Cơ chế local fallback đảm bảo dữ liệu luôn được lưu trên IndexedDB ngay cả khi IPFS hoặc blockchain tạm thời không khả dụng, cùng với retry tự động khi mạng ổn định.
5. **Hỗ trợ đa phương thức xác thực:** Người dùng có thể lựa chọn MetaMask (người dùng Web3 chuyên nghiệp) hoặc đăng nhập Google (người dùng phổ thông) để truy cập hệ thống.

4.9.2 Hạn chế

1. **Thời gian đồng bộ blockchain:** Quá trình xác nhận giao dịch trên Sepolia có thể mất 15–30 giây do phụ thuộc vào tốc độ sinh block, gây ra trải nghiệm chờ đợi cho người dùng.
2. **Phụ thuộc dịch vụ bên thứ ba:** Hệ thống phụ thuộc vào Pinata IPFS Gateway (giới hạn 1 GB ở gói miễn phí) và Firebase Authentication, tạo ra điểm yếu về tính sẵn sàng nếu các dịch vụ này gặp sự cố.
3. **Chi phí gas trên Mainnet:** Mặc dù thử nghiệm trên Sepolia Testnet không mất phí thực, việc triển khai trên Ethereum Mainnet sẽ phát sinh chi phí gas cho mỗi lần cập nhật kết suất.
4. **Chưa hỗ trợ chia sẻ mật khẩu:** Phiên bản hiện tại chưa có tính năng chia sẻ mật khẩu an toàn giữa nhiều người dùng (secure sharing).
5. **Chưa hỗ trợ khôi phục khi quên master password:** Mặc dù có tính năng export nhưng nếu quên master password vẫn không thể khôi phục lại.

4.9.3 Hướng phát triển

1. Tích hợp giải pháp Layer 2 (như Optimism, Arbitrum) để giảm chi phí gas và tăng tốc độ xác nhận giao dịch.
2. Phát triển tính năng chia sẻ mật khẩu an toàn sử dụng mã hóa bất đối xứng (asymmetric encryption).
3. Xây dựng tiện ích mở rộng trình duyệt (browser extension) để tự động điền mật khẩu trên các trang web.
4. Nghiên cứu phát triển dự án sang thành app dùng trên thiết bị di động
5. Nghiên cứu mở rộng tính năng quét rò rỉ HIBP và tích hợp AI hỗ trợ.

Chương 5

Kết luận

Kết luận

Dự án đã nghiên cứu và xây dựng thành công hệ thống quản lý mật khẩu phân tán (Web3 Password Manager DApp), giải quyết triệt để bài toán bảo mật dữ liệu cá nhân trên không gian số. Bằng việc áp dụng kiến trúc mã hóa phía máy khách (Client-side Encryption) với thuật toán tiêu chuẩn AES-256-GCM, kết hợp cùng mô hình lưu trữ lai (Hybrid Storage) giữa cơ sở dữ liệu cục bộ IndexedDB, mạng phân tán IPFS và chuỗi khối Ethereum Sepolia, hệ thống đã đảm bảo tuyệt đối kiến trúc mã hóa phía người dùng. Người dùng hoàn toàn làm chủ khóa mật mã và dữ liệu bản rõ của mình mà không phải phụ thuộc hay tin tưởng vào bất kỳ máy chủ trung gian nào.

Bên cạnh nền tảng bảo mật vững chắc, dự án còn giải quyết thành công rào cản về trải nghiệm người dùng (UX) vốn là điểm yếu của các ứng dụng Web3 truyền thống. Thông qua việc tích hợp cổng xác thực Firebase Google Auth để dẫn xuất ví tất định cục bộ, người dùng phổ thông có thể tiếp cận công nghệ lưu trữ phi tập trung một cách mượt mà và trực quan. Cơ chế đệm dữ liệu tại IndexedDB giúp các thao tác truy xuất diễn ra tức thời, trong khi chính sách quản lý phiên làm việc nghiêm ngặt trên bộ nhớ biến động (RAM-Only) giúp triệt tiêu hoàn toàn các nguy cơ rò rỉ khóa hay tấn công đánh cắp phiên.

Nhìn chung, hệ thống đã đáp ứng đầy đủ các mục tiêu thiết kế và phân tích ban đầu, mang đến một giải pháp quản lý thông tin nhạy cảm an toàn, minh bạch và bền vững. Trong định hướng phát triển tương lai, dự án có tiềm năng mở rộng thông qua việc tích hợp tiêu chuẩn Account Abstraction (ERC-4337) nhằm tài trợ hoàn toàn phí Gas cho người dùng, cũng như phát triển thêm nền tảng ứng dụng trên thiết bị di động để hoàn thiện một hệ sinh thái bảo mật toàn diện.

Tài liệu tham khảo

- [1] Juan Benet. Ipfs - content addressed, versioned, p2p file system. *arXiv preprint arXiv:1407.3561*, 2014.
- [2] Bitwarden. Bitwarden security whitepaper. Bitwarden Documentation, 2024.
- [3] Vitalik Buterin. A next-generation smart contract and decentralized application platform, 2014.
- [4] Google Firebase. Firebase authentication. Firebase Documentation, 2024.
- [5] D. Hardt. The OAuth 2.0 authorization framework. RFC 6749, IETF, 2012.
- [6] Jonathan Katz and Yehuda Lindell. *Introduction to Modern Cryptography*. CRC Press, 3 edition, 2020.
- [7] K. Moriarty, B. Kaliski, and A. Rusch. PKCS #5: Password-based cryptography specification version 2.1. RFC 8018, RFC Editor, 2017.
- [8] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system, 2008.
- [9] National Institute of Standards and Technology. Advanced encryption standard (AES). FIPS Publication 197, National Institute of Standards and Technology, 2001.
- [10] Mark Pilgrim. *HTML5: Up and Running*. " O'Reilly Media, Inc.", 2010.
- [11] Meltem Sönmez Turan, Elaine Barker, William Burr, and Lily Chen. Recommendation for password-based key derivation: Part 1: Storage applications. NIST Special Publication 800-132, National Institute of Standards and Technology, 2010.
- [12] Verizon. 2024 data breach investigations report. Technical report, Verizon Business, 2024.
- [13] W3C. Web cryptography API. W3C Recommendation, 2017.
- [14] Daniel Lowe Wheeler. zxcvbn: Low-budget password strength estimation. In *25th USENIX Security Symposium (USENIX Security 16)*, 2016.

- [15] World Wide Web Consortium (W3C). Indexed database api 2.0. <https://www.w3.org/TR/IndexedDB-2/>, 2021. Accessed: 2026-06-25.
- [16] Pieter Wuille. Bip 32: Hierarchical deterministic wallets. Bitcoin improvement proposal, Bitcoin, 2012. [Truy cập ngày 21-06-2026].
- [17] Guy Zyskind, Oz Nathan, and Alex Pentland. Decentralizing privacy: Using blockchain to protect personal data. In *2015 IEEE Security and Privacy Workshops*, pages 180–184. IEEE, 2015.